

UNIVERSIDAD INDOAMÉRICA

FACULTAD DE INGENIERÍAS

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

TEMA:

MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-8B EN NVIDIA A100 MEDIANTE VLLM

Trabajo de Integración Curricular previo a la obtención del título de Ingeniero en Tecnologías de la Información

Autor

José Andrés Ureta Granda

Tutor

Andrés Xavier Rubio Proaño

QUITO – ECUADOR

2026

AUTORIZACIÓN POR PARTE DEL AUTOR PARA LA CONSULTA, REPRODUCCIÓN PARCIAL O TOTAL, Y PUBLICACIÓN ELECTRÓNICA DEL TRABAJO DE INTEGRACIÓN CURRICULAR

Yo, José Andrés Ureta Granda, declaro ser autor del Trabajo de Titulación con el nombre "Modelado experimental de las fases prefill y decode de Qwen3-8B en NVIDIA A100 mediante vLLM", como requisito para optar al grado de Ingeniero en Tecnologías de la Información y autorizo al Sistema de Bibliotecas de la Universidad Tecnológica Indoamérica para que, con fines netamente académicos, divulgue esta obra a través del Repositorio Digital Institucional (RDI-UTI).

Los usuarios del RDI-UTI podrán consultar el contenido de este trabajo en las redes de información del país y del exterior, con las cuales la Universidad tenga convenios. La Universidad Tecnológica Indoamérica no se hace responsable por el plagio o copia del contenido parcial o total de este trabajo.

Del mismo modo, acepto que los Derechos de Autor, Morales y Patrimoniales sobre esta obra serán compartidos entre mi persona y la Universidad Tecnológica Indoamérica, y que no tramitaré la publicación de este trabajo en ningún otro medio sin autorización expresa de la misma. En caso de que exista el potencial de generación de beneficios económicos o patentes, producto de este trabajo, acepto que se deberán firmar convenios específicos adicionales donde se acuerden los términos de adjudicación de dichos beneficios.

Para constancia de esta autorización, en Quito, Ecuador, a los xx días del mes de xx de 2026, firmo conforme:

Autor: José Andrés Ureta Granda

Firma:

Número de Cédula: 1751545185
Dirección: Pichincha, Quito, Belisario Quevedo, La Gasca
Correo Electrónico: andredks334@gmail.com
Teléfono: 0939116995

APROBACIÓN DEL TUTOR

En mi calidad de Tutor del Trabajo de Titulación "MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-8B EN NVIDIA A100 MEDIANTE VLLM", presentado por José Andrés Ureta Granda, para optar por el Título de Ingeniero en Tecnologías de la Información.

CERTIFICO

Que dicho Trabajo de Titulación ha sido revisado en todas sus partes y considero que reúne los requisitos y méritos suficientes para ser sometido a la presentación pública y evaluación por parte del Tribunal revisor que se designe.

Quito, Ecuador, xx de xx de 2026

Andrés Xavier Rubio Proaño

DECLARACIÓN DE AUTENTICIDAD

Yo, José Andrés Ureta Granda, declaro que los contenidos y los resultados obtenidos en el presente Trabajo de Titulación "Modelado experimental de las fases prefill y decode de Qwen3-8B en NVIDIA A100 mediante vLLM", como requerimiento previo para la obtención del Título de Ingeniero en Tecnologías de la Información, son absolutamente originales, auténticos y personales y de exclusiva responsabilidad legal y académica del autor.

Quito, Ecuador, xx de xx de 2026

José Andrés Ureta Granda
(Número de cédula)

APROBACIÓN DE LECTORES

El Trabajo de Titulación ha sido revisado, aprobado y autorizada su impresión y empastado sobre el tema "MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-8B EN NVIDIA A100 MEDIANTE VLLM", presentado por José Andrés Ureta Granda, previo a la obtención del Título de Ingeniero en Tecnologías de la Información, y reúne los requisitos de fondo y forma para su sustentación.

Quito, Ecuador, xx de xx de 2026

Nombres Apellidos
LECTOR

Nombres Apellidos
LECTORA

DEDICATORIA

(Obligatoria)
La dedicatoria va en la parte inferior derecha de la página.

AGRADECIMIENTO

(Obligatorio)
El agradecimiento va en la parte inferior derecha de la página.

Índice de Contenidos

CAPÍTULO I: Introducción **16**

1.1 Contextualización 17
1.2 Estado del Arte 18
1.2.1 Mecanismos de sistema para la inferencia 18
1.2.2 Sistemas de servicio de modelos de lenguaje 19
1.2.3 Síntesis comparativa y posicionamiento 21
1.3 Planteamiento del Problema 23
1.4 Justificación 24
1.5 Objetivos 25
1.5.1 Objetivo General 25
1.5.2 Objetivos Específicos 26
1.6 Alcance del Proyecto 28
1.6.1 Elementos incluidos 28
1.6.2 Elementos excluidos 28
1.7 Fundamentación Teórica 29
1.7.1 Arquitectura Transformer y atención de consultas agrupadas 30
1.7.2 Fases prefill y decode 30
1.7.3 KV cache 32
1.7.4 Gestión de memoria y planificación en vLLM 32
1.7.5 Representación numérica y cuantización 33
1.7.6 Modelo roofline, MFU y MBU 34
1.7.7 Métricas de desempeño de la inferencia 35

CAPÍTULO II: Metodología **37**

2.1 Diagnóstico de la Situación Actual 37
2.2 Área de Estudio 39
2.3 Metodología de Desarrollo o Implementación 39
2.4 Técnicas e Instrumentos de Recolección de Información 43
2.4.1 Triangulación de la separación prefill/decode 43
2.4.2 Telemetría, energía y control de la contención 44

2.5 Requerimientos del Proyecto 45
2.5.1 Requerimientos Funcionales 45
2.5.2 Requerimientos No Funcionales 45
2.6 Diseño de la Solución 47
2.6.1 Entorno de software congelado 47
2.6.2 Parche de compatibilidad glibc 48
2.7 Justificación Técnica 49
2.7.1 Versiones y backend de atención 49
2.7.2 Techos teóricos y presupuesto de memoria 51

CAPÍTULO III: Implementación, Pruebas y Resultados 53
3.1 Descripción de la Solución 53
3.2 Proceso de Implementación 53
3.2.1 Ejecución del trabajo 30732 53
3.2.2 Rediseño de la ejecución 55
3.3 Pruebas y Validación 56
3.3.1 Prueba de humo y verificación del dispositivo 56
3.3.2 Estabilidad de las mediciones 57
3.3.3 Contraste del presupuesto de memoria 57
3.4 Resultados 58
3.4.1 Fase de prefill 58
3.4.2 Fase de decode 59
3.4.3 Contraste entre fases 60
3.4.4 Telemetría de la GPU 60
3.4.5 Alcance de los resultados piloto 61
3.5 Cronograma y Presupuesto 61

CAPÍTULO IV: Conclusiones y Recomendaciones 62
4.1 Conclusiones 62
4.2 Recomendaciones 62
4.3 Trabajos Futuros 62

REFERENCIAS 63

GLOSARIO 68

ANEXO A: Título del Anexo A 69

Índice de Tablas

1 Comparación de los trabajos revisados en el estado del arte. 22
2 Características de las fases prefill y decode. 31

3 Características de la plataforma experimental. 37
4 Límites de las particiones de Slurm disponibles para el usuario. . . . 38
5 Versión del driver NVIDIA observada en los nodos. 39
6 Área de estudio. 40
7 Fases de ejecución de `bench_pd.sh`. 41
8 Divergencias entre la matriz de la ficha y el barrido piloto implementado. 42
9 Requerimientos funcionales del banco de pruebas. 46
10 Requerimientos no funcionales del banco de pruebas. 46
11 Entorno de software congelado. 47
12 Bibliotecas del entorno con dependencias de glibc 2.32 o superior. . 48
13 Alternativas evaluadas para resolver la incompatibilidad con glibc 2.31. 50

14 Entorno registrado en el trabajo 30732. 54
15 Duración de las ejecuciones del trabajo 30732. 54
16 Experimentos ejecutados por `a100_qwen3_8b.sh`. 56
17 Dispersión de la latencia en las diez iteraciones medidas. 57
18 Latencia y throughput de prefill con lote 1. 58
19 Comparación de las fases frente a sus techos teóricos ($L_{in} = 128$, $B = 1$). 60

Índice de Figuras

Índice de Imágenes

Índice de Ecuaciones

1 Atención de producto escalar escalado 30
2 Tamaño del KV cache por token 32
3 Memoria total del KV cache 32
4 Modelo roofline 34
5 Tiempo por token de salida 35
6 Energía estimada por token 36
7 Latencia de prefill por diferencia de latencias 43
8 Throughput de prefill sin servidor 43
9 TPOT por diferencia de latencias 43
10 Energía estimada a partir de la telemetría 44
11 Energía estimada por token generado 44
12 Volumen de los pesos de Qwen3-8B en BF16 51
13 Techo teórico del decode con lote 1 51
14 Techo teórico del prefill 51
15 KV cache por token de Qwen3-8B 52
16 KV cache de la configuración más exigente 52
17 Presupuesto de memoria de la configuración más exigente 52
18 Utilización de cómputo del prefill 58
19 Costo marginal del prefill por token de entrada 59
20 Utilización del ancho de banda en el decode 59

Índice de Anexos

Anexo A: Título del Anexo A 69

UNIVERSIDAD INDOAMÉRICA

FACULTAD DE INGENIERÍAS

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

TEMA: MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-8B EN NVIDIA A100 MEDIANTE VLLM

AUTOR(A): José Andrés Ureta Granda

TUTOR(A): Andrés Xavier Rubio Proaño

RESUMEN EJECUTIVO

[Redacte el resumen del Trabajo de Integración Curricular en un solo párrafo, con un máximo de 300 palabras. Exponga de manera clara el problema, el objetivo general, la metodología empleada, los principales resultados obtenidos y las conclusiones.]

DESCRIPTORES: *[Incluya de 3 a 4 palabras clave representativas, ordenadas alfabéticamente y separadas por comas.]*

UNIVERSIDAD INDOAMÉRICA

FACULTY OF ENGINEERING

BACHELOR'S DEGREE IN INFORMATION TECHNOLOGY ENGINEERING

TOPIC: EXPERIMENTAL MODELING OF THE PREFILL AND DECODE PHASES OF QWEN3-8B ON NVIDIA A100 USING VLLM

AUTHOR: José Andrés Ureta Granda

ADVISOR: Andrés Xavier Rubio Proaño

ABSTRACT

[Write the English version of the executive summary in a single paragraph, with a maximum of 300 words. Clearly present the problem, general objective, methodology, main results, and conclusions.]

KEYWORDS: *[Include 3 to 4 representative keywords in alphabetical order, separated by commas.]*

CAPÍTULO I

INTRODUCCIÓN

La puesta en servicio de un modelo de lenguaje de gran tamaño (LLM, por sus siglas en inglés) obliga a elegir una configuración de inferencia, y esa elección suele apoyarse en métricas agregadas como los tokens por segundo. Estas métricas resumen en una sola cifra dos fases de ejecución, prefill y decode, que someten a la unidad de procesamiento gráfico (GPU) a exigencias distintas [1]. Una cifra agregada registra, por tanto, el resultado de una configuración sin revelar en qué fase se gana o se pierde rendimiento.

El caso de Qwen3-8B [2] sobre una GPU NVIDIA A100 [3], servido mediante el motor de inferencia vLLM [4], vuelve concreta esta limitación. La ficha institucional del trabajo constata que el modelo puede ejecutarse sin dificultad en una A100 y que, aun así, ninguna configuración resulta óptima para todas las cargas de trabajo. Si la configuración adecuada depende de la carga, elegirla exige conocer cómo responde cada fase a los factores que la definen.

De esta tensión surge la pregunta central del capítulo, que la Sección 1.3 formula de forma explícita: cómo influyen la cuantización, la longitud de contexto y la concurrencia en el comportamiento de las fases prefill y decode de Qwen3-8B, ejecutado mediante vLLM sobre una GPU NVIDIA A100, en términos de rendimiento, memoria, energía y calidad. En esta pregunta, la concurrencia designa el número de solicitudes simultáneas. La longitud de contexto comprende la longitud de entrada, o número de tokens del texto de entrada, y la longitud de salida, o número de tokens generados por solicitud.

El recorrido del capítulo sigue el orden en que se construye esa pregunta y su respuesta. La Sección 1.1 sitúa el trabajo en su contexto técnico e institucional, y la Sección 1.2 revisa la literatura para localizar el vacío que deja abierto. La Sección 1.3 expone las consecuencias de ese vacío para el despliegue y las convierte en la pregunta principal. La Sección 1.4 justifica por qué la pregunta merece respuesta, la Sección 1.5 la descompone en una cadena de objetivos en la que cada uno habilita al siguiente y la Sección 1.6 fija el dominio en que esa respuesta será válida. La Sección 1.7 cierra el capítulo con los fundamentos teóricos que el resto de la memoria requiere.

1.1 Contextualización

Los LLM se han incorporado a asistentes conversacionales, sistemas de consulta documental y flujos de procesamiento automático de texto [5]. Estos modelos se construyen, en su mayoría, sobre la arquitectura Transformer [6]. Una vez entrenado, cada modelo se ejecuta en modo de inferencia ante cada solicitud recibida, de modo que el costo de servirlo se acumula durante toda su vida operativa. La eficiencia de la inferencia condiciona, en consecuencia, la viabilidad de cualquier despliegue.

La inferencia de un LLM se desarrolla en las dos fases ya mencionadas, prefill y decode [1]. La fase prefill procesa el texto de entrada de la solicitud, o prompt, y produce el primer token de salida; la fase decode genera los tokens restantes de uno en uno, de forma autorregresiva. Durante ambas fases, el modelo conserva en la memoria de la GPU las claves y los valores de atención de los tokens previos, estructura que se denomina KV cache. Su funcionamiento se desarrolla en la Sección 1.7; aquí basta señalar que la memoria que ocupa crece con la longitud de contexto y con el número de solicitudes atendidas [4].

Cada fase somete a la GPU a exigencias diferentes, y la configuración de inferencia determina cómo se reparten los recursos entre ambas [1], [7]. En este trabajo, la configuración de inferencia designa la combinación de representación numérica del modelo y parámetros del motor de inferencia con que se atiende una carga de trabajo. De ese reparto dependen dos magnitudes que interesan a quien despliega el servicio: la latencia percibida por los usuarios y el throughput, entendido como el número de tokens procesados por segundo. Una configuración que favorece una de ellas puede perjudicar la otra [7]; de ahí que la distinción entre fases ocupe un lugar central en los trabajos revisados en la Sección 1.2.

El presente trabajo de fin de grado (TFG) se enmarca en el proyecto LLM-CoDesign, que caracteriza la inferencia de modelos de lenguaje considerando conjuntamente el modelo, la carga de trabajo, el hardware y el software. Según la ficha institucional del TFG, el proyecto abarca múltiples modelos, GPU, motores de inferencia y métricas durante cinco años. Un alcance de esa amplitud necesita un primer caso controlado sobre el cual validar la metodología antes de extenderla. Este TFG reduce el alcance del proyecto a ese caso y constituye su primer caso de estudio.

El caso seleccionado combina el modelo Qwen3-8B, perteneciente a la familia Qwen3 [2], con una GPU NVIDIA A100 y el motor de inferencia vLLM [4]. La A100 se comercializa en variantes de 40 GB y 80 GB de memoria [3]. Fijado el caso, queda por establecer qué sabe ya la literatura sobre la inferencia de LLM en una GPU y qué deja sin resolver para esta combinación.

1.2 Estado del Arte

La distinción entre prefill y decode, presentada en la sección anterior, atraviesa la literatura sobre inferencia de LLM. Esa literatura se concentra en mecanismos de sistema orientados a aprovechar mejor la GPU: la planificación de solicitudes, la gestión de memoria, los kernels de atención, el tratamiento diferenciado de las fases y la cuantización. La Sección 1.2.1 revisa seis trabajos representativos de esas líneas y señala, en cada caso, qué aportan a la pregunta central y qué dejan abierto. La Sección 1.2.2 examina cómo esos mecanismos se integran en los sistemas que sirven los modelos a los usuarios, y la Sección 1.2.3 sintetiza la comparación y sitúa el presente trabajo frente a ella.

1.2.1 Mecanismos de sistema para la inferencia

Yu et al. [8] proponen Orca, un motor de inferencia que planifica la ejecución a nivel de iteración en lugar de a nivel de solicitud. Con esta estrategia, el sistema incorpora solicitudes nuevas al lote y retira las finalizadas en cada paso de generación. Orca reserva, sin embargo, el espacio del KV cache de cada solicitud en bloques contiguos dimensionados de antemano, y esa reserva desperdicia una parte de la memoria disponible [4]. La planificación por iteración mejora así el aprovechamiento del cómputo, mientras deja abierto el de la memoria.

Kwon et al. [4] abordan ese problema con PagedAttention, un mecanismo que gestiona el KV cache en bloques de tamaño fijo, de forma análoga a la memoria virtual paginada de un sistema operativo. La gestión paginada reduce la fragmentación de la memoria y permite compartir bloques entre solicitudes. Sobre este mecanismo, los autores construyen vLLM, que combina la gestión paginada con la planificación a nivel de iteración, conocida como batching continuo. vLLM es el motor de inferencia empleado en este trabajo; su evaluación original, sin embargo, no separa el comportamiento de prefill y decode.

Dao et al. [9] desarrollan FlashAttention, un algoritmo de atención exacta que reduce los accesos a la memoria de alto ancho de banda de la GPU mediante el procesamiento por bloques. El trabajo pone de relieve el peso del movimiento de datos entre niveles de memoria en el costo de la atención. Esta perspectiva resulta pertinente para el presente TFG, porque el límite que impone la memoria frente al que impone el cómputo es precisamente lo que distingue a una fase de la otra [1].

Agrawal et al. [7] estudian el compromiso entre throughput y latencia y observan que la incorporación del prefill de solicitudes nuevas a un lote interrumpe la generación de las solicitudes que se encuentran en decode. Como respuesta, proponen

Sarathi-Serve, un motor de inferencia que divide el prefill en fragmentos (chunked prefill) y los planifica sin detener la generación en curso. La coexistencia de ambas fases en un mismo lote tiene una consecuencia directa para este trabajo: condiciona la manera de medirlas por separado.

Zhong et al. [1] llevan la separación más lejos con DistServe, un motor de inferencia que asigna prefill y decode a GPU distintas para eliminar la interferencia entre fases. Su análisis caracteriza prefill como una fase limitada por el cómputo y decode como una fase limitada por el ancho de banda de memoria. Los autores evalúan cada fase con una métrica propia: el TTFT (*time to first token*), tiempo hasta el primer token, asociado a prefill; y el TPOT (*time per output token*), tiempo por token de salida, asociado a decode. Este trabajo adopta ambas métricas, aunque su caso de una sola GPU no admite la separación física que DistServe propone.

Lin et al. [10] proponen AWQ, un método de cuantización de solo pesos a baja precisión que protege los pesos más relevantes a partir de la distribución de las activaciones. La representación de los pesos con 4 bits (AWQ INT4) reduce la memoria ocupada por el modelo frente al formato de coma flotante de 16 bits BF16. La memoria que libera la cuantización queda disponible para el KV cache, de modo que la representación numérica puede alterar el comportamiento de ambas fases. Por esa razón, este TFG compara las dos representaciones numéricas.

1.2.2 Sistemas de servicio de modelos de lenguaje

Los trabajos anteriores proponen mecanismos que, en la práctica, no operan de forma aislada: se integran en un servidor de inferencia, el sistema que recibe las solicitudes de los usuarios, decide cuándo y con qué recursos se atiende cada una y devuelve los tokens generados. Ejecutar el modelo directamente desde una biblioteca de aprendizaje profundo resuelve el cálculo de una solicitud, pero no las exigencias de latencia y throughput de un servicio que atiende a muchos usuarios; los sistemas de inferencia especializados surgen para cubrir esa distancia [11]. Tres funciones distinguen a un servidor de inferencia de la ejecución directa: la planificación de las peticiones, el agrupamiento de las solicitudes concurrentes y la gestión de la memoria que ocupa el KV cache.

La planificación determina en qué orden y en qué momento se atiende cada petición. La latencia que percibe un usuario no depende solo del tiempo de cálculo de su solicitud, sino también del tiempo que espera hasta ser atendida y de las solicitudes con las que comparte la GPU. Por ello, la manera en que el servidor planifica y agrupa las peticiones condiciona directamente la latencia del servicio [12]. Esa dependencia explica que el TTFT incluya, en un servidor con carga, el tiempo de espera en cola,

como precisa la Sección 1.7.7.

El agrupamiento de solicitudes concurrentes se apoya en la asimetría entre las dos fases: prefill está limitado por el cómputo, mientras que decode está limitado por el ancho de banda de memoria [13]. Como cada paso de decode lee todos los pesos del modelo para producir un solo token por solicitud, reunir varias solicitudes en un mismo paso reparte esa lectura entre ellas. La forma de reunirlas es decisiva. El agrupamiento estático forma un lote y lo mantiene hasta que terminan todas sus solicitudes: las que finalizan antes dejan su espacio sin usar, y las que llegan después esperan al lote siguiente [8]. El agrupamiento continuo, introducido por Orca [8], recompone el lote en cada iteración del modelo, de modo que incorpora solicitudes nuevas en cuanto hay capacidad y retira las finalizadas sin esperar al resto. La contrapartida es que prefill y decode conviven en el mismo lote e interfieren entre sí [7]; el mecanismo que mejora el aprovechamiento de la GPU es, por tanto, el mismo que complica la medición separada de las fases.

Con el agrupamiento continuo, la concurrencia deja de ser un dato fijo de la carga y se convierte en una variable que el planificador gestiona activamente: decide cuántas solicitudes admite, cuáles aplaza y cómo redistribuye los recursos cuando escasean [14]. Para este trabajo, esa gestión obliga a interpretar con cuidado el nivel de concurrencia. El benchmark fija el número de solicitudes simultáneas que se envían al servidor, pero el número de solicitudes que el motor procesa en cada iteración depende de su planificador y de la memoria disponible.

La gestión de la memoria del KV cache es la tercera función, y está acoplada a las dos anteriores. Una reserva contigua dimensionada de antemano, como la de Orca, desperdicia memoria y limita el número de solicitudes que caben en la GPU [4]. La gestión paginada de vLLM asigna el KV cache en bloques de tamaño fijo a medida que crece cada secuencia [4], con el funcionamiento que detalla la Sección 1.7.4. Su consecuencia para el servicio es que la memoria libre, y no una reserva fija por solicitud, determina cuántas solicitudes pueden agruparse. Planificación, agrupamiento y memoria quedan así ligados entre sí, y los tres dependen de la longitud de contexto y de la concurrencia, dos de los factores que este trabajo hace variar.

vLLM no es el único sistema que combina estas funciones. Entre las alternativas figura SGLang [15], y los propios Sarathi-Serve y DistServe, revisados en la Sección 1.2.1, son motores de inferencia completos. La elección de vLLM en este trabajo responde a cuatro razones. La primera es que la ficha institucional lo fija como el componente de software *S* de la configuración experimental. La segunda es que integra en un mismo motor la gestión paginada del KV cache, el agrupamiento continuo y el chunked prefill, que son precisamente los mecanismos cuya interac-

ción con las fases se pretende observar. La tercera es que expone esos mecanismos mediante parámetros de configuración, descritos en la Sección 1.7.4, lo que permite controlarlos y documentarlos. La cuarta es que su código abierto permite fijar y citar la versión exacta empleada, condición de la reproducibilidad. Comparar vLLM con otros motores queda, por su parte, fuera del alcance, como establece la Sección 1.6.2.

1.2.3 Síntesis comparativa y posicionamiento

La Tabla 1 resume, para cada trabajo citado, el sistema o modelo evaluado, el hardware empleado, si separa las fases prefill y decode, las métricas que reporta y su limitación para el presente TFG; su última fila describe el diseño de este trabajo. Ninguno de los trabajos revisados caracteriza la combinación de Qwen3-8B, A100 y vLLM, y todos ellos son anteriores a la publicación de la familia Qwen3 [2].

La última fila de la Tabla 1 plantea una objeción inmediata. DistServe [1] y Splitwise [16] ya separan prefill y decode y ya los miden por separado; Splitwise caracteriza, además, la potencia de cada fase [16]. Cabe preguntarse, entonces, qué añade una nueva caracterización por fases. La respuesta no está en qué fases se miden, sino en dónde se ejecutan. Ambos trabajos desagregan las fases sobre recursos distintos: asignan prefill a unas GPU y decode a otras, y diseñan el sistema para que cada grupo de recursos atienda una sola fase. Esa desagregación es la solución de un operador hiperescalar, que dispone de numerosas GPU interconectadas y puede dedicar una parte de ellas a cada fase. Una universidad o un laboratorio sin acceso a esa infraestructura sirve el modelo, en cambio, desde una única GPU, en la que ambas fases comparten el cómputo, el ancho de banda de memoria y el espacio disponible para el KV cache. En ese régimen, la interferencia que la desagregación elimina forma parte de las condiciones de funcionamiento, y el reparto de recursos entre fases no lo decide la topología del clúster, sino el planificador del motor y sus parámetros de configuración. Una sola GPU compartida no es, por tanto, una versión reducida de un sistema desagregado, sino un régimen distinto, con compromisos propios que los trabajos desagregados no necesitan resolver. Este TFG caracteriza el comportamiento de ambas fases precisamente en ese régimen.

La literatura revisada explica, por tanto, por qué prefill y decode se comportan de forma distinta, pero no ofrece una caracterización experimental de ambas fases cuando comparten una sola GPU, y menos aún para el caso seleccionado. La Sección 1.3 examina las consecuencias de ese vacío para quien debe configurar el servicio.

Tabla 1: Comparación de los trabajos revisados en el estado del arte.

Trabajo	**Año**	**Sistema o modelo evaluado**	**Hardware**	**¿Separa prefill y decode?**	**Métricas reportadas**	**Limitación para este TFG**
Flash-Attention [9]	2022	Kernel de atención; entrenamiento de BERT y GPT-2	GPU A100	No aplica: optimiza un kernel	Tiempo de ejecución y memoria	Optimiza un cálculo aislado, no el servicio
Orca [8]	2022	Motor Orca; modelos de la familia GPT-3	Varias GPU A100	No: ambas fases en el mismo lote	Latencia y throughput	Reserva contigua del KV cache
Pope et al. [13]	2023	Inferencia de PaLM	TPU v4	Sí, en el análisis: prefill limitado por cómputo, decode por memoria	Latencia y MFU	Otro acelerador; modelo repartido entre muchos chips
vLLM [4]	2023	Motor vLLM; modelos OPT y LLaMA	GPU A100	No	Throughput a latencia normalizada	Evaluación agregada, sin desglose por fases
AWQ [10]	2024	Cuantización de pesos; modelos LLaMA y OPT	GPU de escritorio y embebidas	No	Perplejidad, precisión y aceleración	No mide el efecto por fases bajo carga concurrente
Sarathi-Serve [7]	2024	Motor Sarathi-Serve; varias familias de modelos	GPU A100 y A40	En parte: distingue las fases, pero las mezcla en el lote	TTFT, latencia entre tokens y capacidad	La mezcla de fases condiciona su medición separada
DistServe [1]	2024	Motor DistServe; modelos OPT	Varias GPU A100	Sí: GPU distintas para cada fase	TTFT, TPOT y goodput	Requiere varias GPU
SGLang [15]	2024	Motor SGLang, alternativo a vLLM	GPU A10G y A100	No	Throughput y latencia	Comparar motores queda fuera del alcance
Splitwise [16]	2024	Separación de fases; modelos BLOOM y LLaMA-2	Servidores DGX-A100 y DGX-H100	Sí: máquinas distintas para cada fase	TTFT, latencia entre tokens, throughput, potencia y costo	Requiere un clúster de varias máquinas
Este TFG	**2026**	Qwen3-8B en BF16 y AWQ INT4, servido con vLLM; controla longitud de entrada, longitud de salida y concurrencia	Una única GPU A100 (variante documentada)	Sí: ambas fases comparten una sola GPU	TTFT, TPOT, throughput, latencia total, memoria máxima, utilización, potencia media y energía estimada por token	Validez limitada a Qwen3-8B, A100 y vLLM; energía estimada por telemetría

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia

1.3 Planteamiento del Problema

El vacío identificado en el estado del arte tiene un efecto práctico. La ficha institucional del trabajo constata que la selección de una configuración de inferencia suele basarse en métricas agregadas, como los tokens por segundo, sin distinguir el comportamiento de las fases prefill y decode. Una métrica agregada resume en una sola cifra dos procesos con exigencias distintas sobre la GPU [1]. El resultado es una visión incompleta del sistema, que registra los compromisos entre fases sin explicarlos.

Esa visión incompleta deja sin explicar varios fenómenos relevantes para el despliegue. No permite justificar por qué una configuración mejora el throughput y empeora la latencia interactiva, ni en qué casos la cuantización permite aumentar la concurrencia. Tampoco aclara cómo influye la longitud de entrada sobre el TTFT, ni cómo afecta el crecimiento del KV cache a la latencia durante decode. Cada uno de estos fenómenos se origina en una fase concreta; por ello, ninguna cifra agregada alcanza a explicarlo.

La consecuencia práctica es la dificultad para elegir una configuración adecuada según el uso previsto. Según la ficha institucional, un servicio de chat interactivo, una consulta documental basada en recuperación (RAG, por sus siglas en inglés) y un procesamiento por lotes imponen prioridades diferentes. Una configuración apropiada para un escenario puede resultar, por tanto, inadecuada para otro.

El caso de Qwen3-8B en una A100 ilustra la situación. La ficha institucional constata que el modelo puede ejecutarse sin dificultad en esa GPU, pero esta posibilidad no garantiza la existencia de una configuración universalmente óptima: el rendimiento depende de la carga de trabajo, de la concurrencia y de la representación numérica del modelo. Para esta combinación concreta no se dispone, además, de una caracterización experimental que separe ambas fases, como muestra la Tabla 1. Ese vacío, trasladado al despliegue, constituye el problema del trabajo.

La caracterización que falta puede concretarse en preguntas específicas que la vuelven medible. Las dos primeras atañen a cada fase por separado: cómo cambia el TTFT cuando aumenta la longitud de entrada y cómo cambia el TPOT cuando aumenta la concurrencia. La tercera examina cómo influye la longitud de salida en la latencia y en la energía.

Otras tres preguntas se ocupan de la memoria y de la capacidad del sistema: cuánta memoria ocupan las representaciones numéricas BF16 y AWQ INT4, en qué condiciones la cuantización mejora la capacidad del sistema y qué configuraciones producen saturación o errores por falta de memoria. Las dos últimas completan el recorrido: cuáles son los principales cuellos de botella de cada fase y con qué error

estima un modelo predictivo básico el TTFT y el TPOT a partir de las variables experimentales delimitadas en la Sección 1.6.1.

Responder a estas preguntas exige una secuencia de productos: un entorno en el que ejecutar el modelo, una carga controlada, un conjunto de mediciones, un análisis por fases y un modelo predictivo. La Sección 1.4 expone por qué estas preguntas merecen respuesta, y la Sección 1.5 organiza esos productos como una cadena de objetivos. Todas ellas convergen en una sola pregunta, con la que se cierra el planteamiento.

En consecuencia, el problema descrito se formula mediante la siguiente pregunta principal: ¿cómo influyen la cuantización, la longitud de contexto y la concurrencia en el comportamiento de las fases prefill y decode de Qwen3-8B, ejecutado mediante vLLM sobre una GPU NVIDIA A100, en términos de rendimiento, memoria, energía y calidad?

1.4 Justificación

La pregunta se justifica, en primer lugar, desde el punto de vista técnico. El análisis separado de prefill y decode permite explicar los compromisos entre latencia, throughput y memoria que las métricas agregadas solo registran. Williams et al. [17] proponen el modelo roofline, que relaciona el rendimiento alcanzable con los límites de cómputo y de ancho de banda de memoria del procesador. Este marco ofrece una referencia para situar cada fase respecto a los límites de la A100 y sustenta el análisis de cuellos de botella que persigue el objetivo específico OE4 (Sección 1.5.2).

Desde el punto de vista operativo, el trabajo orienta la elección de configuraciones para los tres escenarios de uso presentados en la Sección 1.3. Según la ficha institucional, el chat interactivo prioriza un TTFT y un TPOT bajos; la consulta documental requiere un equilibrio entre longitud de entrada, latencia y memoria; y el procesamiento por lotes prioriza el throughput y la eficiencia energética. Una caracterización por fases permite relacionar cada una de esas prioridades con la fase que la gobierna.

La utilidad del trabajo se materializa en el prototipo Qwen-A100 Profiler, que la ficha institucional define como su producto tecnológico. Esta herramienta automatiza la ejecución de los experimentos, recolecta las métricas, almacena los resultados de forma estructurada y estima el TTFT y el TPOT mediante el modelo predictivo que persigue el objetivo específico OE5. El conjunto de datos experimental y los procedimientos de reproducción completan los entregables reutilizables, de modo que el estudio puede repetirse sin reconstruir la metodología.

Esos entregables definen a los beneficiarios del trabajo. El beneficiario directo es el proyecto LLM-CoDesign, que obtiene su primer caso de estudio y una metodología de caracterización aplicable a otras combinaciones de modelo, hardware y software. Los beneficiarios indirectos son los equipos técnicos que despliegan LLM en infraestructura propia y necesitan criterios cuantitativos para configurar el servicio.

La justificación exige, por último, que el trabajo sea factible. En el plano técnico, Qwen3-8B cabe en la memoria de una única A100, y las herramientas necesarias (vLLM, las herramientas de telemetría NVML y DCGM, y las bibliotecas de análisis de datos) son de libre acceso. El trabajo no requiere entrenar el modelo, desarrollar kernels de GPU ni modificar internamente vLLM. En el plano operativo, el plan de trabajo comprende doce semanas con una dedicación de 15 a 20 horas semanales, y la factibilidad depende de disponer de acceso controlado a la GPU durante las campañas de medición. Justificada la pregunta, la Sección 1.5 la traslada a una cadena de objetivos.

1.5 Objetivos

Las preguntas formuladas en la Sección 1.3 se responden mediante un objetivo general y cuatro objetivos específicos. Sus enunciados reformulan los de la ficha institucional aprobada: conservan su contenido, expresan cada objetivo con un solo verbo e integran el diseño del benchmark en el objetivo de medición, del que es el instrumento.

1.5.1 Objetivo General

Identificar el efecto de la cuantización, la longitud de contexto y la concurrencia sobre el comportamiento de las fases prefill y decode de Qwen3-8B, servido mediante vLLM sobre una única GPU NVIDIA A100, en términos de rendimiento, memoria, energía y calidad, mediante la medición controlada de ambas fases y un modelo predictivo básico del TTFT y del TPOT, como base para la selección de configuraciones de inferencia según el escenario de uso.

El objetivo general traslada la pregunta principal a una acción verificable. Retoma los mismos tres factores y las mismas cuatro dimensiones que la pregunta nombra, precisa los dos medios con que se alcanza, la medición controlada de ambas fases y el modelo predictivo, y declara su finalidad: servir de base a la elección de configuraciones en los escenarios de uso descritos en la Sección 1.3.

1.5.2 Objetivos Específicos

OE1. Levantar un entorno reproducible de ejecución de Qwen3-8B mediante vLLM sobre una A100, mediante la fijación y documentación de las versiones de CUDA, del controlador de NVIDIA, de PyTorch y de vLLM en un contenedor o entorno versionado, como garantía de que las diferencias observadas entre configuraciones puedan atribuirse a los factores estudiados y no a cambios del entorno.

OE2. Medir el TTFT, el TPOT, el throughput, la memoria GPU, la utilización y el consumo energético estimado, mediante un benchmark parametrizable con niveles controlados de longitud de entrada, longitud de salida y concurrencia sobre las representaciones BF16 y AWQ INT4, como base empírica del análisis por fases.

OE3. Analizar por separado el comportamiento de las fases prefill y decode, mediante el contraste de las mediciones con los límites de cómputo y de ancho de banda de memoria de la A100 según el modelo roofline, con vistas a la localización de los principales cuellos de botella de cada fase.

OE4. Construir un modelo predictivo básico del TTFT y del TPOT, a partir de las variables experimentales y de las relaciones identificadas en el análisis por fases, como soporte de la estimación y la recomendación de configuraciones del prototipo Qwen-A100 Profiler dentro de los niveles estudiados.

Los cuatro objetivos específicos se leen como una cadena y no como una lista. Los párrafos siguientes explican, para cada uno, qué vacío cierra, por qué condiciona al objetivo que le sigue y qué aporta al objetivo general.

OE1 cierra la ausencia de un entorno controlado en el que medir. La ficha institucional exige congelar y documentar las versiones de CUDA, del controlador de NVIDIA, de PyTorch y de vLLM, de modo que los experimentos puedan reconstruirse. Sin esa condición, ninguna diferencia observada entre configuraciones podría atribuirse a los factores estudiados en lugar de a cambios del entorno. Por esa razón, OE1 es condición necesaria de OE2: dos mediciones solo son comparables si se realizan sobre el mismo entorno. Al objetivo general, OE1 le aporta la reproducibilidad de la medición.

OE2 cierra dos vacíos a la vez: la distancia entre la pregunta y los datos, y la ausencia de mediciones para esta combinación, señalada en la Tabla 1. Las preguntas específicas varían la longitud de entrada, la longitud de salida y la concurrencia, y ningún entorno genera por sí solo cargas con esos niveles controlados. El benchmark

parametrizable produce esas cargas de forma sistemática sobre el entorno de OE1; es, por tanto, el instrumento de la medición y no un fin en sí mismo. La cuantización se introduce al ejecutar el modelo en sus dos representaciones numéricas. El TTFT y el TPOT reflejan cada fase por separado; el throughput, la memoria GPU, la utilización y el consumo energético cubren las dimensiones de rendimiento, memoria y energía del objetivo general. El consumo energético se obtiene como estimación a partir de la telemetría interna de la GPU, según precisa la Sección 1.6.1. OE2 es condición necesaria de OE3, porque una métrica solo puede atribuirse a un factor, y a una fase, si la carga que la produjo está controlada.

OE3 cierra el vacío central del planteamiento: la explicación de los compromisos que las métricas agregadas registran sin explicar. Separar prefill y decode permite atribuir cada efecto observado a la fase que lo produce, y el contraste con los límites del modelo roofline, descrito en la Sección 1.7.6, permite localizar en cuál de ellas aparece cada cuello de botella. Es condición necesaria de OE4, porque el modelo predictivo estima el TTFT y el TPOT por separado, y cada estimación exige conocer qué variables gobiernan la fase correspondiente. Con OE3 se completa el primero de los dos medios que nombra el objetivo general.

OE4 cierra el paso de la descripción a la anticipación. Un modelo predictivo básico estima el TTFT y el TPOT de una configuración a partir de las variables experimentales, sin necesidad de ejecutarla, dentro de los niveles estudiados. Como último eslabón, OE4 no habilita otro objetivo específico; alimenta, en cambio, la estimación que realiza el prototipo Qwen-A100 Profiler descrito en la Sección 1.4. Con él se completa el segundo medio del objetivo general.

Para OE4, la ficha institucional establece como meta referencial un error porcentual absoluto medio (MAPE) inferior al 15–20 % en las métricas principales. Esta meta es ajustable después de los experimentos piloto. No constituye, por tanto, un compromiso cerrado sobre la precisión del modelo predictivo.

Leídos en conjunto, los cuatro objetivos forman una cadena: el entorno de OE1 sostiene las mediciones de OE2, que alimentan el análisis por fases de OE3 y, a través de él, el modelo predictivo de OE4. La dimensión de calidad que el objetivo general incluye se aborda mediante la comparación limitada entre BF16 y AWQ INT4 que precisa la Sección 1.6.1. Queda por precisar hasta dónde llega la respuesta, cometido de la Sección 1.6.

1.6 Alcance del Proyecto

La respuesta a la pregunta central vale únicamente dentro de un dominio experimental definido. La Sección 1.6.1 precisa ese dominio y la Sección 1.6.2 enumera lo que queda fuera de él.

1.6.1 Elementos incluidos

La unidad de análisis es la configuración experimental $C = (M, W, H, S)$. En ella, M es el modelo Qwen3-8B; W, las características de la carga de trabajo; H, una GPU NVIDIA A100; y S, vLLM con sus parámetros de configuración. La variante exacta de la A100, de 40 GB u 80 GB, se documenta como parte del entorno experimental, porque su capacidad de memoria condiciona los niveles máximos de longitud de contexto y concurrencia alcanzables.

Sobre esa unidad se varían cuatro factores, que constituyen las variables experimentales. La cuantización se evalúa a través de la representación numérica, con dos niveles: BF16 y AWQ INT4. La longitud de entrada toma cuatro niveles, de 128, 512, 2048 y 8192 tokens; la longitud de salida, dos niveles, de 64 y 256 tokens; y la concurrencia, cuatro niveles, de 1, 4, 8 y 16 solicitudes simultáneas. La combinación de los niveles produce $2 \times 4 \times 2 \times 4 = 64$ configuraciones, y con cinco repeticiones por configuración la campaña comprende alrededor de 320 ejecuciones medidas. Las configuraciones se agrupan además en tres tipos de carga de trabajo: chat corto, consulta documental y contexto largo.

Las métricas registradas son el TTFT, el TPOT, el throughput, la latencia total de la solicitud, la memoria máxima utilizada, la utilización de la GPU, la potencia media y la energía estimada por token. La energía se presenta como una estimación basada en la telemetría interna de la GPU, obtenida mediante NVML y DCGM. No constituye una medición eléctrica calibrada externamente, salvedad que la ficha institucional exige hacer explícita.

El alcance incluye una comparación limitada de calidad entre BF16 y AWQ INT4 para identificar posibles degradaciones asociadas a la cuantización. Incluye también el prototipo Qwen-A100 Profiler en su versión de producto mínimo viable, con una recomendación básica de configuraciones para los tres escenarios de uso. Definido lo que el trabajo abarca, la Sección 1.6.2 fija lo que excluye.

1.6.2 Elementos excluidos

Las exclusiones siguientes acotan el trabajo a un caso controlado e impiden que sus conclusiones se extiendan más allá de lo medido:

- Entrenamiento completo o fine-tuning del modelo.
- Comparación con otras familias de LLM.
- Evaluación de múltiples GPU y ejecución multi-GPU.
- Paralelismo tensorial o de pipeline.
- Comparación entre varios motores de inferencia.
- Modelos Mixture-of-Experts o multimodales.
- Speculative decoding.
- Longitudes de entrada superiores a 8192 tokens en la campaña principal.
- Predicción del comportamiento en otro hardware.
- Simulación de sistemas distribuidos.
- Desarrollo completo del asesor multiobjetivo de LLM-CoDesign.

Las conclusiones del trabajo serán válidas únicamente para la combinación de Qwen3-8B, NVIDIA A100 y vLLM. No se formularán afirmaciones de superioridad general sobre otras plataformas o modelos. Delimitado así el dominio de la respuesta, la Sección 1.7 expone los fundamentos teóricos que los capítulos siguientes requieren.

1.7 Fundamentación Teórica

Las secciones anteriores emplean conceptos que solo se han mencionado: las fases prefill y decode, el KV cache, la cuantización y los límites de cómputo y de memoria de la GPU. Esta sección los desarrolla en el orden en que intervienen durante la inferencia. Primero se describe la arquitectura del modelo y las dos fases de su ejecución; después, la estructura de memoria que las conecta y los mecanismos con que vLLM la gestiona; por último, la cuantización, el modelo roofline y las métricas con que se caracteriza cada fase. Los valores numéricos para Qwen3-8B y la A100 se calculan en el Capítulo II; aquí se establecen las relaciones de las que esos valores se derivan.

1.7.1 Arquitectura Transformer y atención de consultas agrupadas

Qwen3-8B es un modelo Transformer de solo decodificador [2]. La arquitectura Transformer apila capas idénticas, cada una formada por un bloque de autoatención y una red de alimentación directa [6]. En un modelo de solo decodificador, cada token solo puede atender a los tokens que lo preceden, de modo que el texto se genera de forma autorregresiva: cada token nuevo se calcula a partir de todos los anteriores.

El bloque de autoatención proyecta la representación de cada token en tres vectores: una consulta (Q), una clave (K) y un valor (V). La salida de la atención se obtiene con la Ecuación 1, donde d_k es la dimensión de las claves [6]. El producto $QK^\top$ compara la consulta de cada token con las claves de todos los tokens previos, y el resultado pondera los valores correspondientes. La atención multicabeza repite este cálculo en paralelo con varias proyecciones independientes, denominadas cabezas.

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1}$$

La Ecuación 1 tiene una consecuencia directa para la inferencia: para generar un token, el modelo necesita las claves y los valores de todos los tokens anteriores. Recalcularlos en cada paso sería redundante, porque no cambian; por ello se conservan en memoria, en la estructura que la Sección 1.7.3 describe.

La atención de consultas agrupadas (GQA, por sus siglas en inglés) reduce el volumen de esa estructura. En lugar de asignar claves y valores propios a cada cabeza de consulta, GQA reparte las cabezas de consulta en grupos que comparten un mismo par de cabezas de clave y valor [18]. Qwen3-8B emplea GQA con 32 cabezas de consulta y 8 cabezas de clave y valor [2], de modo que almacena cuatro veces menos claves y valores que un modelo equivalente con atención multicabeza convencional. Esta reducción es la que hace manejable el KV cache de Qwen3-8B en una única GPU con contextos largos.

1.7.2 Fases prefill y decode

La inferencia de una solicitud se divide en dos fases con un patrón de cálculo distinto [1]. En la fase prefill, el modelo procesa los L_{in} tokens del prompt en una sola pasada. Como todos ellos se conocen de antemano, se calculan en paralelo: cada capa opera sobre una matriz de L_{in} filas, y los pesos leídos desde la memoria se reutilizan para todos los tokens. La fase termina con el primer token de salida y con las claves y los valores del prompt ya almacenados.

En la fase decode, el modelo genera los $L_{out} - 1$ tokens restantes, uno por paso. Cada paso procesa un único token por solicitud, porque el siguiente depende de él. El paso exige, sin embargo, leer todos los pesos del modelo y las claves y los valores acumulados, aunque el cálculo realizado con ellos sea pequeño. Cuando el motor agrupa B solicitudes en decode, un mismo paso reutiliza los pesos para las B solicitudes, pero cada una aporta un solo token.

La diferencia entre ambas fases se expresa mediante la intensidad aritmética, definida en la Sección 1.7.6 como el número de operaciones realizadas por byte leído de memoria. En las capas lineales, un modelo de N parámetros realiza aproximadamente $2N$ operaciones de coma flotante por token, una multiplicación y una suma por parámetro. En BF16 lee $2N$ bytes de pesos por pasada. La intensidad aritmética del prefill crece, por tanto, de forma aproximadamente proporcional a L_{in}, mientras que la del decode crece con el número de solicitudes B del lote. Por esa razón, el prefill tiende a quedar limitado por el cómputo y el decode por el ancho de banda de memoria [1]. La Tabla 2 resume las diferencias.

Tabla 2: Características de las fases prefill y decode.

Característica	**Prefill**	**Decode**
Tokens procesados por solicitud	Los L_{in} del prompt, en una pasada	Uno por paso, durante $L_{out} - 1$ pasos
Intensidad aritmética	Crece con L_{in}	Crece con el tamaño del lote B
Recurso limitante típico	Cómputo	Ancho de banda de memoria
Efecto sobre el KV cache	Lo crea para el prompt	Lo amplía en un token por paso
Métrica asociada	TTFT	TPOT

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia a partir de [1], [7]

El costo de la atención añade un término que depende de la longitud de contexto. En el prefill, cada token del prompt atiende a los anteriores, de modo que el cálculo de la atención crece con el cuadrado de L_{in}; con prompts largos, este término gana peso frente al de las capas lineales. FlashAttention reduce los accesos a memoria de este cálculo procesándolo por bloques, sin alterar su resultado [9]. En el decode, cada paso lee las claves y los valores de todo el contexto acumulado, por lo que el volumen leído por paso crece con la longitud de la secuencia y con el número de solicitudes del lote.

1.7.3 KV cache

El KV cache es la estructura en la que el motor de inferencia conserva, en la memoria de la GPU, las claves y los valores de cada token ya procesado. Lo crea la fase prefill y lo amplía cada paso de la fase decode. Su tamaño por token se obtiene con la Ecuación 2, donde n_{capas} es el número de capas, n_{KV} el número de cabezas de clave y valor, d_h la dimensión de cada cabeza y b el número de bytes por elemento. El factor 2 corresponde a las claves y los valores.

$$m_{KV} = 2 \times n_{capas} \times n_{KV} \times d_h \times b \tag{2}$$

La memoria total que ocupa el KV cache crece de forma lineal con el número de tokens almacenados, como recoge la Ecuación 3. Para B solicitudes con longitudes de secuencia $L_1, \ldots, L_B$, la suma depende a la vez de la longitud de contexto y de la concurrencia, las dos variables que este trabajo hace variar.

$$M_{KV} = m_{KV} \sum_{i=1}^{B} L_i \tag{3}$$

La memoria de la GPU se reparte entre los pesos del modelo, las activaciones y el KV cache. Una vez cargados los pesos, la memoria restante fija cuántos tokens pueden almacenarse y, con ello, cuántas solicitudes pueden atenderse de forma simultánea. El valor de m_{KV} para Qwen3-8B y el presupuesto de memoria de la configuración más exigente se calculan en la Ecuación 15 y siguientes del Capítulo II.

1.7.4 Gestión de memoria y planificación en vLLM

vLLM gestiona el KV cache mediante PagedAttention [4]. Este mecanismo divide el KV cache de cada solicitud en bloques de un número fijo de tokens, que no necesitan ocupar posiciones contiguas en la memoria. Una tabla de bloques asocia las posiciones lógicas de cada secuencia con los bloques físicos que las almacenan, de forma análoga a la tabla de páginas de la memoria virtual de un sistema operativo. Los bloques se asignan a medida que la secuencia crece, de modo que el desperdicio queda limitado al último bloque, que puede estar parcialmente ocupado. La misma indirección permite que varias solicitudes compartan los bloques de un prefijo común [4].

Cuando el KV cache se llena, vLLM recurre a la expulsión (preemption) de solicitudes: libera los bloques de una o más de ellas y, cuando vuelve a haber memoria,

las reanuda trasladando sus bloques a la memoria de la CPU o recalculándolos [4]. La expulsión mantiene el servicio en funcionamiento, pero alarga la latencia de las solicitudes afectadas. Por ello, el número de expulsiones es un indicador de que la configuración opera en el límite de la memoria.

La planificación de solicitudes sigue el esquema de batching continuo introducido por Orca [8]. El planificador decide el contenido del lote en cada iteración del modelo y no una vez por solicitud: incorpora solicitudes nuevas en cuanto hay capacidad y retira las finalizadas sin esperar al resto del lote. En consecuencia, un mismo lote puede contener solicitudes en prefill y solicitudes en decode.

Esa convivencia genera interferencia entre fases. Un prefill largo ocupa la GPU durante una iteración costosa y retrasa el paso siguiente de todas las solicitudes en decode, lo que eleva su TPOT [7]. El chunked prefill atenúa este efecto: divide el prefill en fragmentos y limita el número de tokens procesados en cada iteración, de modo que el decode avanza sin detenerse mientras se completa el prefill [7]. La contrapartida es un TTFT mayor para la solicitud cuyo prefill se fragmenta.

El comportamiento de estos mecanismos se controla mediante parámetros de configuración del motor. Los principales son la fracción de la memoria de la GPU que vLLM puede reservar (`gpu_memory_utilization`), el número máximo de solicitudes simultáneas en un lote (`max_num_seqs`) y el número máximo de tokens por iteración (`max_num_batched_tokens`), que acota el tamaño de los fragmentos del chunked prefill. vLLM combina, además, la ejecución del modelo mediante grafos CUDA con la ejecución inmediata (*eager*) de PyTorch; la opción `enforce_eager` desactiva los grafos y ejecuta siempre en modo inmediato.[1] La configuración base de estos parámetros se fija en el Capítulo II.

1.7.5 Representación numérica y cuantización

La representación numérica determina cuántos bytes ocupa cada peso y, con ello, el volumen que se lee de memoria en cada pasada. BF16 es un formato de coma flotante de 16 bits que conserva el rango dinámico de la coma flotante de 32 bits y reduce la precisión de la mantisa [19]. La A100 ejecuta operaciones BF16 en sus Tensor Cores [3]. En BF16, cada peso ocupa 2 bytes.

La cuantización representa los pesos con menos bits. AWQ es un método de cuantización de solo pesos: los pesos se almacenan en enteros de 4 bits (INT4), mientras que las activaciones y el cálculo se mantienen en coma flotante de 16

[1] Definiciones tomadas de la documentación de los parámetros en el código fuente de vLLM 0.20.0 (`vllm/config/cache.py`, `scheduler.py` y `model.py`), disponible en `https://github.com/vllm-project/vllm/tree/v0.20.0/vllm/config`. Consultado el 24 de octubre de 2026.

bits [10]. El método parte de la observación de que una pequeña fracción de los pesos es determinante para la calidad del modelo, y que esa fracción puede identificarse a partir de la magnitud de las activaciones y no de los propios pesos. Para protegerla, AWQ escala los canales correspondientes antes de cuantizar, de modo que su error de cuantización relativo disminuye [10].

El efecto esperable de la cuantización difiere entre fases. En el decode, limitado por el ancho de banda de memoria, leer 4 bits por peso en lugar de 16 reduce el volumen leído por paso, que es el recurso limitante [10]. En el prefill, limitado por el cómputo, esa reducción no alivia el recurso limitante, y los pesos deben convertirse a coma flotante antes de operar con ellos. La memoria que libera la cuantización queda, además, disponible para el KV cache y, con ello, para atender más solicitudes. La reducción de bits puede degradar la calidad de las respuestas, por lo que el alcance incluye la comparación limitada de calidad descrita en la Sección 1.6.1. Que estos efectos se produzcan, y en qué magnitud, para Qwen3-8B en la A100 es una de las preguntas que el trabajo responde experimentalmente.

1.7.6 Modelo roofline, MFU y MBU

El modelo roofline acota el rendimiento alcanzable por un cálculo a partir de dos límites del procesador: su rendimiento máximo de cómputo y el ancho de banda de su memoria [17]. El modelo caracteriza cada cálculo por su intensidad aritmética I, el cociente entre las operaciones de coma flotante que realiza y los bytes que transfiere desde la memoria. El rendimiento alcanzable P es el menor de los dos límites, como expresa la Ecuación 4, donde P_{pico} es el rendimiento máximo de cómputo y BW el ancho de banda de memoria.

$$P = \text{mín}\left(P_{pico},\ I \times BW\right) \tag{4}$$

Los dos límites se cruzan en el punto de equilibrio $I^* = P_{pico}/BW$. Un cálculo con $I < I^*$ queda limitado por la memoria, y uno con $I > I^*$, por el cómputo. Para la A100 de 40 GB, con 312 TFLOP/s en BF16 denso y 1555 GB/s de ancho de banda [3], el punto de equilibrio se sitúa en torno a 200 operaciones por byte.

Este marco traduce a cifras la diferencia descrita en la Sección 1.7.2. Como la intensidad del prefill crece con L_{in}, un prompt de varios cientos de tokens supera el punto de equilibrio y queda limitado por el cómputo. Como la intensidad del decode crece con B, un lote de pocas solicitudes queda muy por debajo del punto de equilibrio y el paso queda limitado por la lectura de los pesos y del KV cache. El

modelo proporciona así una cota superior del rendimiento de cada fase, cuyo cálculo para Qwen3-8B se realiza en la Sección 2.7.2.

Dos métricas expresan qué fracción de esa cota se alcanza. La utilización de FLOPs del modelo (MFU, por sus siglas en inglés) es el cociente entre el throughput observado y el máximo teórico que alcanzaría el sistema operando a P_{pico}, contando solo las operaciones que el modelo exige [20]; resulta adecuada para el prefill, limitado por el cómputo. La utilización del ancho de banda del modelo (MBU) es el cociente entre los bytes por segundo que el modelo lee efectivamente, pesos y KV cache incluidos, y BW; resulta adecuada para el decode, limitado por la memoria [21]. Un valor bajo de la métrica correspondiente indica que la fase no aprovecha el recurso que la limita, y orienta la búsqueda del cuello de botella hacia otras causas, como la planificación o la interferencia entre fases.

1.7.7 Métricas de desempeño de la inferencia

La caracterización por fases exige métricas que separen la contribución de cada una a la latencia percibida. El TTFT es el tiempo que transcurre desde el envío de la solicitud hasta la recepción del primer token; refleja la fase prefill y, en un servidor con carga, también el tiempo de espera en cola. El TPOT es el tiempo medio entre tokens de salida consecutivos de una solicitud y refleja la fase decode [1]. La latencia extremo a extremo (E2E) es el tiempo total desde el envío hasta el último token. Las tres se relacionan mediante la Ecuación 5, que define el TPOT a partir de las otras dos.

$$\mathrm{TPOT} = \frac{\mathrm{E2E} - \mathrm{TTFT}}{L_{out} - 1} \tag{5}$$

El TPOT es un promedio por solicitud. La latencia entre tokens (ITL, por sus siglas en inglés) registra, en cambio, cada intervalo individual entre dos tokens consecutivos. Su distribución revela pausas que el promedio oculta, como las que produce un prefill largo al interferir con el decode [7].

El throughput mide la capacidad del sistema y se expresa en tokens de salida por segundo o en solicitudes atendidas por segundo. El goodput restringe ese recuento a las solicitudes que cumplen unos objetivos de nivel de servicio (SLO) sobre el TTFT y el TPOT [1]. A diferencia del throughput, el goodput penaliza una configuración que procesa muchos tokens a costa de una latencia inaceptable para el usuario.

La energía se estima a partir de la potencia que informa la telemetría interna de la GPU, mediante NVML y DCGM. La energía consumida durante un intervalo es

la integral de la potencia en ese intervalo, y la energía por token se obtiene con la Ecuación 6, donde $p(t)$ es la potencia instantánea entre los instantes t_0 y t_1, y N_{tok} el número de tokens generados en ese intervalo.

$$E_{tok} = \frac{1}{N_{tok}} \int_{t_0}^{t_1} p(t)\, dt \tag{6}$$

Como precisa la Sección 1.6.1, esta energía es una estimación basada en la telemetría y no una medición eléctrica calibrada. Con estas métricas, los mecanismos descritos en esta sección y las cotas del modelo roofline, el Capítulo II establece el método con que se miden las dos fases.

CAPÍTULO II

METODOLOGÍA

2.1 Diagnóstico de la Situación Actual

El método de este trabajo no se diseñó en abstracto, sino sobre una infraestructura concreta cuyas características imponen condiciones a la medición. Por esa razón, el capítulo parte de un diagnóstico de la plataforma disponible antes de describir el diseño experimental. Las limitaciones identificadas aquí reaparecen en las secciones siguientes como el porqué de buena parte de las decisiones metodológicas.

La plataforma experimental es el clúster de computación de alto rendimiento de CEDIA (`hpc.cedia.edu.ec`), gestionado con el planificador de trabajos Slurm [22]. Se eligió esta plataforma porque es la que ofrece acceso a la GPU NVIDIA A100 que fija el caso de estudio, y no por preferencia frente a otras infraestructuras. La Tabla 3 resume sus características. Los datos de rendimiento de la GPU proceden de la hoja de especificaciones del fabricante [3].

Tabla 3: Características de la plataforma experimental.

Elemento	**Valor**
Clúster	HPC CEDIA, planificador Slurm
Nodos de cómputo GPU	`compute-0-1` y `compute-0-2`, con 8 NVIDIA A100-SXM4-40GB cada uno
Nodo interactivo	`compute-0-0`, con 1 A100-40GB, 6 particiones MIG `a100_2g.10gb` y 2 particiones MIG `a100_1g.5gb`
CPU por nodo	256 núcleos
RAM por nodo	≈960 GB (nodos GPU); ≈1 TB (nodo interactivo)
Memoria de la GPU	40 960 MiB
Límite de potencia de la GPU	400 W
Ancho de banda HBM2	1555 GB/s
Rendimiento BF16 con Tensor Cores	312 TFLOPS (denso)
Sistema operativo	Ubuntu 20.04.6, glibc 2.31
Almacenamiento de trabajo	NFS compartido (`$HOME`)

Elaborado por: José Andrés Ureta Granda

Fuente: Inspección de los nodos del clúster HPC CEDIA; hoja de especificaciones de la A100 [3]

La variante de la GPU es la A100-SXM4 de 40 GB, dato que la ficha institucional exige documentar porque la capacidad de memoria condiciona los niveles máximos de longitud de contexto y concurrencia. El sistema operativo de los nodos es Ubuntu 20.04.6, mientras que la ficha sugiere Ubuntu 22.04 o equivalente. La diferencia no es una elección del trabajo, sino una condición del clúster que no puede modificarse. Su consecuencia más importante es la versión de glibc, la 2.31, que origina el problema de compatibilidad tratado en la Sección 2.6.2.

El nodo interactivo ofrece particiones MIG (*Multi-Instance GPU*), que dividen una A100 en instancias aisladas de menor capacidad. Estas particiones quedan excluidas de la campaña de medición por dos razones. La primera es que su memoria resulta insuficiente para alojar Qwen3-8B. La segunda es que NVML no expone la potencia por instancia MIG, lo que impediría atribuir el consumo energético que exige el tercer objetivo específico.[1]

La exclusión de MIG obliga, además, a una precaución de identificación. En un trabajo asignado a una partición MIG, `nvidia-smi` muestra la A100 de 40 GB que la contiene y no la instancia efectiva. Por ello, el dispositivo real se identifica con `nvidia-smi -L`, `torch.cuda.get_device_name()` o `torch.cuda.mem_get_info()`, y no con la salida general de `nvidia-smi`.

El planificador impone los límites recogidos en la Tabla 4. A ellos se suman dos restricciones de la calidad de servicio (QoS) `normal` asignada al usuario: solo se admite un trabajo en ejecución simultánea y no es posible reservar un nodo completo. La partición por defecto es `interactive`, que solo ofrece particiones MIG. En consecuencia, omitir la partición en la solicitud conduce a un recurso inservible para este experimento.

Tabla 4: Límites de las particiones de Slurm disponibles para el usuario.

Partición	**Tiempo máx.**	**GPU máx.**	**CPU máx.**	**RAM máx.**
`interactive` (por defecto)	1 día	—	—	—
`gpu`	2 días	2	160	128 G
`gpu-max`	1 día	4	128	256 G
`gpu-dev`	4 días	1	16	—

Elaborado por: José Andrés Ureta Granda

[1] La documentación de NVML define `nvmlDeviceGetPowerUsage` y `nvmlDeviceGetTotalEnergyConsumption` sobre la GPU completa y sus circuitos asociados, sin declarar soporte para los manejadores de instancia MIG; en cambio, consultas como la de memoria sí indican que la información por instancia se obtiene con esos manejadores. NVIDIA, *NVML API Reference Guide*, disponible en `https://docs.nvidia.com/deploy/nvml-api/`. Consultado el 24 de octubre de 2026.

Fuente: Configuración de Slurm del clúster HPC CEDIA

Dos limitaciones adicionales afectan directamente a la validez de las mediciones de latencia. La primera es que los nodos son compartidos con otros usuarios y su carga de CPU varía durante la ejecución, como se detalla en la Sección 2.4.2. La segunda es que los nodos GPU no comparten la misma versión del driver, según muestra la Tabla 5. Como el planificador asigna el nodo en cada trabajo, dos mediciones de la misma configuración pueden ejecutarse con drivers distintos y no ser directamente comparables.

Tabla 5: Versión del driver NVIDIA observada en los nodos.

Fecha	Nodo	Driver
20 de octubre	`compute-0-1`	535.161.08
22 de octubre	`compute-0-2`	575.51.03
24 de octubre	`compute-0-1` (trabajo 30732)	535.161.08
27 de octubre	`compute-0-1` (trabajo 31157)	535.161.08

Elaborado por: José Andrés Ureta Granda

Fuente: Registros `entorno.txt` de los trabajos ejecutados

El diagnóstico deja establecidas, por tanto, las condiciones de partida: una A100 de 40 GB sin reserva exclusiva del nodo, un sistema operativo con glibc 2.31, un driver que puede cambiar entre ejecuciones y un único trabajo simultáneo. Antes de describir cómo se diseñó el experimento sobre esas condiciones, la Sección 2.2 delimita el área de estudio.

2.2 Área de Estudio

La Tabla 6 sitúa el trabajo en su dominio académico y precisa el objeto de estudio. Su contenido se deriva de la ficha institucional aprobada.

Con el objeto de estudio delimitado, la Sección 2.3 describe el método con que se caracteriza.

2.3 Metodología de Desarrollo o Implementación

El trabajo sigue un método experimental cuantitativo organizado en las tres fases que establece la ficha institucional: preparación del entorno, benchmark controlado e instrumentación. Se adoptó esa estructura porque la ficha es el documento normativo del trabajo y porque separa con claridad lo que debe congelarse, lo que se varía y lo que se mide. La preparación del entorno se desarrolla en la Sección 2.6. Esta sección describe cómo se organiza la medición y cómo las restricciones del clúster

Tabla 6: Área de estudio.

Dominio:	Sistemas de inferencia de modelos de lenguaje de gran tamaño sobre infraestructura de computación de alto rendimiento
Línea de investigación:	Diseño, desarrollo e integración de sistemas basados en arquitecturas web, móviles y sistemas embebidos
Sublínea de investigación:	Optimización del rendimiento.
Campo:	Tecnologías de la información y la comunicación (TIC)
Área:	Caracterización y modelado del rendimiento de la inferencia de LLM en GPU
Aspectos:	Comportamiento de las fases prefill y decode; TTFT, TPOT, throughput, memoria, utilización y energía estimada; efecto de la cuantización, la longitud de contexto y la concurrencia
Objeto de estudio:	Modelo Qwen3-8B servido con vLLM sobre una GPU NVIDIA A100-SXM4-40GB del clúster HPC CEDIA
Periodo de estudio:	*[Indique las fechas o el intervalo de tiempo que comprende el estudio.]*

condicionaron su diseño.

Las restricciones de Slurm determinaron tres decisiones de diseño. La primera es la ejecución secuencial de las configuraciones: con un único trabajo simultáneo, no es posible paralelizarlas entre GPU. La segunda es la elección de la partición `gpu-dev` en lugar de `gpu`. El 24 de octubre, `sbatch --test-only` estimaba en `gpu` un inicio para el 31 de octubre pese a haber GPU libres, mientras que en `gpu-dev` la estimación era inmediata; `gpu-dev` ofrece, además, 4 días de tiempo máximo de ejecución frente a 2.

La tercera decisión es la reutilización del motor de inferencia. Arrancar vLLM con Qwen3-8B tiene un costo fijo elevado: en el trabajo 30732, la inicialización del motor tardó entre 4 y 24 minutos, dominada por la compilación con `torch.compile`, y cada ejecución completa costó entre 29 y 75 minutos, como detalla la Sección 3.2.1. Reiniciar el motor para cada configuración multiplicaría ese costo por el número de configuraciones dentro de un único trabajo con tiempo limitado. Por ello, la fase en línea levanta el servidor una sola vez y recorre todas las configuraciones contra él.

La medición se automatiza con el script `bench_pd.sh`, que organiza cada trabajo en las cinco fases de la Tabla 7. Se automatizó todo el recorrido en un único script porque cada trabajo de Slurm debe ser autosuficiente: el investigador no interviene entre fases, y cualquier paso manual introduciría variaciones no registradas. Cada fase deja sus salidas en un directorio propio del trabajo, `resultados/`*jobid*`/`, de modo que toda medición queda asociada al identificador del trabajo que la produjo.

Tabla 7: Fases de ejecución de `bench_pd.sh`.

Fase	Contenido	Salidas
0	Registro del nodo, la GPU, las versiones y la carga; arranque del monitor de telemetría	`entorno.txt`, `gpu_monitor.csv`
1	Prueba de humo con el modelo objetivo	`smoke.txt`, `smoke.err`
2	Barrido sin servidor (`vllm bench latency`)	`lat_in*_out*_bs*.json`
3	Barrido en línea (`vllm serve` y `vllm bench serve`)	`serve_in*_out*_c*.json`, `metrics_servidor.txt`
4	Agregación y tablas	`RESUMEN.md`

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia a partir de `bench_pd.sh`

El orden de las fases responde a un criterio de economía del tiempo de GPU. La fase 0 registra el entorno antes de medir, porque el driver y la carga del nodo pueden cambiar entre trabajos. La fase 1 verifica que el modelo carga y genera texto antes de comprometer el resto del tiempo asignado, de modo que un fallo del entorno

se detecta antes de invertir horas de GPU en un barrido inválido. Las fases 2 y 3 aportan dos perspectivas complementarias de la misma medición, que la Sección 2.4 explica.

El barrido implementado tiene carácter de barrido piloto y no reproduce todavía la matriz de la ficha. La ficha prevé este paso en la mitigación de su riesgo R6: ejecutar un piloto que cronometre cada configuración y renegociar la matriz con el tutor a partir de ese dato. La Tabla 8 declara todas las divergencias entre ambas matrices. El piloto explora un espacio más amplio en longitud de entrada y concurrencia para calibrar la matriz definitiva, que se acordará por escrito con el tutor antes de la campaña.

Tabla 8: Divergencias entre la matriz de la ficha y el barrido piloto implementado.

Variable	**Ficha aprobada**	**Barrido piloto (`bench_pd.sh`)**
Variante numérica	BF16 y AWQ INT4	Solo BF16
Longitud de entrada	128, 512, 2048 y 8192	128, 512, 1024, 2048 y 4096
Longitud de salida	64 y 256	1 y 129 (sin servidor); 128 y 256 (en línea)
Concurrencia	1, 4, 8 y 16	1, 2, 4, 8, 16, 32 y 64
Repeticiones	5	10 iteraciones (sin servidor)

Elaborado por: José Andrés Ureta Granda
Fuente: Ficha institucional ITI-TFG-B26-0017 y `bench_pd.sh`

Dos de esas divergencias son huecos que el piloto debe cerrar antes de fijar la matriz. El primero es el escenario de 8192 tokens, que no se mide. Es la configuración límite en memoria, según el cálculo de la Sección 2.7.2, y por tanto la que decide si la matriz de la ficha es ejecutable en una A100 de 40 GB; puede exigir, además, desactivar los grafos CUDA con `--enforce-eager`. El segundo hueco es la variante AWQ INT4 [10], que constituye la mitad de la comparación exigida por el objetivo general y cuyo riesgo asociado, la disponibilidad y fiabilidad de un checkpoint cuantizado, sigue abierto.

Quedan otras dos decisiones pendientes que la memoria declara en lugar de omitir. La ficha prevé aleatorizar el orden de las configuraciones para reducir el efecto de la temperatura, la caché y el estado previo del sistema, y el material del piloto no documenta esa aleatorización; la campaña definitiva deberá incorporarla. Por otra parte, el riesgo R4 de la ficha advierte que el chunked prefill puede mezclar ambas fases en un mismo lote [7], y la decisión de desactivarlo o documentarlo sigue abierta. Con la organización de la medición descrita, la Sección 2.4 precisa qué se mide y con qué instrumentos.

2.4 Técnicas e Instrumentos de Recolección de Información

La técnica central del trabajo es la medición instrumentada de la inferencia. Las métricas por fase son el TTFT (*time to first token*), que caracteriza el prefill, y el TPOT (*time per output token*), que caracteriza el decode. Ninguna de las dos se observa directamente en un único instrumento sin supuestos, y por esa razón se triangulan con tres métodos independientes. Si los tres métodos convergen, la medición es robusta; si divergen, la magnitud de la discrepancia acota el error de cada uno y constituye un hallazgo que merece discusión.

2.4.1 Triangulación de la separación prefill/decode

El primer método obtiene las fases por diferencia de latencias en el modo sin servidor, con `vllm bench latency`. Con una longitud de salida de un token, el tiempo medido corresponde esencialmente al prefill; con 129 tokens, se añaden 128 pasos de decode. Se eligió la salida de 129 tokens para que la diferencia contenga exactamente 128 pasos de decode. Las Ecuaciones 7 a 9 formalizan el procedimiento, donde $T(n)$ es la latencia medida con n tokens de salida, L_{in} la longitud de entrada y B el tamaño de lote.

$$T_{prefill} \approx T(1) \tag{7}$$

$$X_{prefill} = \frac{L_{in} \times B}{T(1)} \tag{8}$$

$$\text{TPOT} = \frac{T(129) - T(1)}{128} \tag{9}$$

La ventaja de este método es que aísla las fases sin instrumentación interna del motor. Su limitación es que $T(1)$ no es prefill puro: incluye una pequeña fracción de decode y el sobrecoste de arranque de la iteración. Por ello, la Ecuación 7 es una aproximación y su error debe contrastarse con los otros dos métodos.

El segundo método recoge las métricas de cliente en el modo en línea, con `vllm bench serve` contra un servidor `vllm serve`. La herramienta reporta el TTFT, el TPOT y el ITL (*inter-token latency*, latencia entre tokens consecutivos) con los percentiles 50, 90 y 99. Se ejecuta con la opción `--ignore-eos` para que cada solicitud genere la longitud de salida pedida aunque el modelo emita antes el token de

fin de secuencia; sin ella, solicitudes de la misma configuración tendrían longitudes distintas y su TPOT no sería comparable. Este método ofrece la perspectiva del usuario, y su limitación es que el TTFT incluye el tiempo de espera en cola del servidor, además del prefill.

El tercer método lee las métricas internas del servidor en el endpoint `/metrics` de vLLM. Los histogramas `request_prefill_time_seconds` y `request_decode_time_seconds` separan las fases según la contabilidad interna del motor. Es la medida más directa y por eso sirve de referencia para validar los dos métodos anteriores. Su limitación es que depende de cómo el motor define el inicio y el fin de cada fase, definición que no es observable desde fuera y que puede variar entre versiones de vLLM.[2]

2.4.2 Telemetría, energía y control de la contención

La telemetría de la GPU se registra con un monitor de `nvidia-smi` que muestrea cada 2 s durante toda la ejecución y escribe `gpu_monitor.csv`. Se recurrió a `nvidia-smi` porque está disponible en los nodos y expone la memoria, la utilización, la temperatura y la potencia que exige el tercer objetivo específico. La ficha prevé DCGM para la campaña principal y Nsight Systems solo en un subconjunto representativo, por el volumen de datos y el sobrecoste del perfilado.

La energía se calculará a partir de esa telemetría con las Ecuaciones 10 y 11, definidas en la ficha. En ellas, P_i es la potencia de la muestra i, Δt el periodo de muestreo, E_{idle} la energía consumida en reposo durante el mismo intervalo y N_{tokens} el número de tokens generados. La energía así obtenida es una estimación basada en la telemetría interna de la GPU y no una medición eléctrica calibrada externamente. La ventana de integración y la medición de E_{idle} en cada campaña quedan por fijar en el protocolo de energía.

$$E = \sum_{i=1}^{n} P_i \, \Delta t \tag{10}$$

$$E_{token} = \frac{E_{ejecucion} - E_{idle}}{N_{tokens}} \tag{11}$$

[2]Las opciones de `vllm bench serve` (`--ignore-eos`, `--metric-percentiles`) y los histogramas `vllm:request_prefill_time_seconds` y `vllm:request_decode_time_seconds` se documentan en el código fuente de vLLM 0.20.0 (`vllm/benchmarks/serve.py` y `vllm/v1/metrics/loggers.py`), disponible en `https://github.com/vllm-project/vllm/tree/v0.20.0`. Consultado el 24 de octubre de 2026.

La contención del nodo es una variable de confusión, porque el TTFT y el TPOT dependen de que la CPU alimente a la GPU sin demora. Su magnitud quedó documentada durante la preparación: el 22 de octubre a las 19:34, `compute-0-2` registraba un load average de 317,33, 337,72 y 214,73 en las ventanas de 1, 5 y 15 minutos, sobre 256 núcleos. Esa carga equivale a una sobresuscripción de 1,24 veces en la ventana de un minuto y de 1,32 veces en la de cinco, frente a 0,84 veces quince minutos antes. La carga se duplicó, por tanto, mientras el trabajo estaba en ejecución, y nada garantiza que no vuelva a ocurrir.

Como el usuario no puede reservar un nodo completo, la contención no puede eliminarse y se controla de tres formas. La primera es registrar `/proc/loadavg` al inicio de cada trabajo en `entorno.txt`, para que la carga quede asociada a cada medición. La segunda es el monitor de `nvidia-smi` descrito, que permite detectar intervalos de baja utilización atribuibles a la CPU. La tercera es ejecutar repeticiones suficientes para estimar la varianza y detectar ejecuciones contaminadas, en lugar de confiar en una medición única.

La diferencia de driver entre nodos de la Tabla 5 se controla con el mismo principio de registro. Cada ejecución guarda en `entorno.txt` el nodo y la versión del driver vigente en el momento de medir. La campaña definitiva deberá ejecutarse en un mismo nodo o, si el planificador no lo permite, reportar el driver como covariable en el análisis. Definidos los instrumentos y sus controles, la Sección 2.5 fija los requerimientos que debe cumplir el banco de pruebas que los integra.

2.5 Requerimientos del Proyecto

Los requerimientos del banco de pruebas se derivan de los objetivos específicos y de las condiciones identificadas en el diagnóstico. Se formularon como requerimientos verificables porque el banco es un instrumento de medición, y un instrumento sin especificación no puede validarse. Las Tablas 9 y 10 los recogen junto con la razón de cada uno.

2.5.1 Requerimientos Funcionales

Elaborado por: José Andrés Ureta Granda
Fuente: Especificación del proyecto (`SPEC.md`, §11.1)

2.5.2 Requerimientos No Funcionales

Elaborado por: José Andrés Ureta Granda

Tabla 9: Requerimientos funcionales del banco de pruebas.

ID	Requerimiento	Razón
RF1	Desplegar vLLM con una configuración parametrizada en YAML y verificar su salud mediante `/health`	Evita medir contra un servidor que aún no está listo
RF2	Generar prompts sintéticos de longitud exacta en tokens con el tokenizador de Qwen3	La longitud de entrada es una variable experimental y debe controlarse en tokens, no en caracteres
RF3	Ejecutar la carga en modo sin servidor (lote fijo) y en modo en línea (llegadas Poisson, cliente asíncrono con streaming)	Sostiene los métodos 1 y 2 de la triangulación
RF4	Registrar los tiempos por petición y por token con marcas de tiempo de alta resolución en el cliente	Permite calcular TTFT, TPOT e ITL desde la perspectiva del usuario
RF5	Muestrear en paralelo la telemetría de la GPU y las métricas Prometheus de vLLM	Sostiene el método 3, la estimación de energía y el control de la contención
RF6	Persistir cada corrida con sus metadatos: versiones, hardware, configuración, semilla y marca de tiempo	Hace comparables mediciones separadas por días o por cambios de driver
RF7	Ajustar los modelos y generar automáticamente las tablas y figuras para LaTeX	Elimina la transcripción manual de resultados a la memoria

Tabla 10: Requerimientos no funcionales del banco de pruebas.

ID	Requerimiento	Razón
RNF1 Reproducibilidad	Versiones fijadas, semillas fijas, contenedor o lockfile y scripts idempotentes	El primer objetivo específico exige un entorno reproducible
RNF2 Aislamiento	GPU dedicada sin otros procesos durante las mediciones, verificada con `nvidia-smi`	Otro proceso en la GPU contaminaría latencia y energía; la contención de CPU, que no puede aislarse, se controla según la Sección 2.4.2
RNF3 Precisión de medición	Reloj monotónico (`time.perf_counter_ns`), warm-up descartado y repeticiones suficientes	Un reloj ajustable o una primera ejecución en frío sesgarían el TTFT
RNF4 Trazabilidad	Cada figura o tabla de la memoria referencia el identificador de la corrida de origen	Permite auditar cualquier cifra hasta su dato crudo
RNF5 Costo	La campaña completa cabe en el presupuesto de horas de GPU	Con un único trabajo simultáneo, el tiempo de GPU es el recurso escaso

Fuente: Especificación del proyecto (`SPEC.md`, §11.2)

El requerimiento RNF1 es el que más condiciona el diseño, porque obliga a congelar un entorno que el clúster no ofrece listo para usar. La Sección 2.6 describe cómo se construyó ese entorno.

2.6 Diseño de la Solución

El diseño de la solución comprende dos elementos: el entorno de software congelado sobre el que se ejecutan todas las mediciones y el parche que hace posible ese entorno en los nodos del clúster. Ambos responden al requerimiento RNF1, porque una medición solo es reproducible si lo es el entorno que la produce.

2.6.1 Entorno de software congelado

La Tabla 11 recoge los componentes del entorno y sus versiones exactas. Se congelaron la revisión del modelo y el commit de la rueda de vLLM, y no solo sus números de versión, porque un mismo número de versión puede corresponder a binarios o pesos distintos. Las dependencias completas quedan en el archivo `req-congelado.txt`, que acompaña al repositorio. El entorno se construyó como un entorno Python versionado, una de las tres vías de reconstrucción que admite la ficha junto con Docker y Apptainer.

Tabla 11: Entorno de software congelado.

Componente	**Versión**
Python	3.11 (módulo `python/3.11`, miniconda)
PyTorch	2.11.0+cu129
CUDA runtime	12.9
vLLM	0.20.0+cu129
Procedencia de la rueda	`wheels.vllm.ai`, commit `88d34c6409e9fb3c7b8ca0c04756f061d2099eb1`
Backend de atención	FlashAttention 2
Modelo	`Qwen/Qwen3-8B`, revisión `b968826d9c46dd6066d109eabc6255188de91218`
Caché de modelos	`HF_HOME=~/tfg-jose-ureta/hf-cache`
Dependencias completas	`req-congelado.txt`

Elaborado por: José Andrés Ureta Granda
Fuente: Entorno instalado en el clúster HPC CEDIA

La razón de cada versión, en particular la de vLLM 0.20.0 y la del backend FlashAttention 2, se expone en la Sección 2.7. Antes es necesario resolver un problema que impedía cargar esa versión de vLLM en los nodos.

2.6.2 Parche de compatibilidad glibc

La rueda `vllm-0.20.0+cu129` se distribuye con la etiqueta `manylinux_2_31`, que declara compatibilidad con glibc 2.31 o superior [23]. Sin embargo, dos de sus extensiones nativas, `_C.abi3.so` y `_moe_C.abi3.so`, enlazan cuatro símbolos con la versión `GLIBC_2.34`: `dlopen`, `dlclose`, `dlsym` y `pthread_once`. En los nodos, con glibc 2.31, el resultado es un `ImportError` al cargar vLLM. La etiqueta de la rueda es, por tanto, incorrecta.

El diagnóstico consistió en inspeccionar con `objdump -T` todas las bibliotecas compartidas del entorno virtual. Solo cuatro declaran dependencias de glibc 2.32 o superior, y la Tabla 12 indica cuáles se cargan efectivamente en el clúster. Las dos que no se cargan quedan fuera del problema: una implementa FlashAttention 3, que solo existe para Hopper, y la otra está compilada para Python 3.12. El parche debe actuar, en consecuencia, solo sobre las dos extensiones del núcleo de vLLM.

Tabla 12: Bibliotecas del entorno con dependencias de glibc 2.32 o superior.

Biblioteca	**¿Se carga en el clúster?**
`vllm/_C.abi3.so`	Sí: núcleo de vLLM
`vllm/_moe_C.abi3.so`	Sí
`vllm/vllm_flash_attn/_vllm_fa3_C.abi3.so`	No: FlashAttention 3 es exclusiva de Hopper
`vllm/third_party/deep_gemm/_C.cpython-312-*.so`	No: compilada para Python 3.12

Elaborado por: José Andrés Ureta Granda
Fuente: Inspección con `objdump -T` del entorno virtual

Los cuatro símbolos no corresponden a funcionalidad ausente en glibc 2.31. En glibc 2.34 las bibliotecas `libdl` y `libpthread` se consolidaron dentro de `libc`; en glibc 2.31 esos mismos símbolos existen, pero residen en `libdl.so.2` y `libpthread.so.0`.[3] La incompatibilidad es, por tanto, un artefacto de haber compilado la rueda en un sistema más moderno, y basta con indicar al cargador dinámico dónde encontrar los símbolos.

La solución se implementó en el script `parche_glibc.sh`, que aplica tres operaciones. Primero, elimina la versión de los cuatro símbolos con `patchelf --clear-symbol-version` y añade `libdl.so.2` y `libpthread.so.0` como dependencias explícitas con `--add-needed`.[4] Segundo, marca como `VER_FLG_WEAK`

[3] GNU C Library, notas de la versión 2.34 (archivo `NEWS`, apartado *Major new features*): toda la funcionalidad de `libpthread`, `libdl`, `libutil` y `libanl` se integró en `libc`. Disponible en `https://sourceware.org/git/?p=glibc.git;a=blob_plain;f=NEWS;hb=glibc-2.34`. Consultado el 24 de octubre de 2026.

[4] patchelf, utilidad para modificar ejecutables y bibliotecas ELF; ambas opciones se describen

la entrada GLIBC_2.34 de la sección .gnu.version_r mediante parche_glibc_weak.py, porque el paso anterior no basta: el cargador sigue rechazando la biblioteca mientras esa entrada exista sin la bandera. Tercero, verifica que import vllm._C e import vllm._moe_C funcionen.

El script es idempotente y conserva los binarios originales como *.so.orig, de modo que aplicarlo varias veces no altera el resultado y el cambio es reversible. Debe reaplicarse tras cada reinstalación del entorno virtual, y por eso bench_pd.sh lo invoca automáticamente si detecta que vLLM no se importa. Así, ningún trabajo de medición depende de que el parche se haya aplicado a mano.

El parche no altera el comportamiento numérico de vLLM. Solo reubica dónde busca el cargador dinámico cuatro símbolos de carga de bibliotecas y sincronización; no modifica kernels, precisión numérica ni planificación de solicitudes. Aun así, constituye una modificación del binario distribuido, y el entorno resultante no es bit a bit idéntico al que se obtendría con pip install vllm==0.20.0 en un sistema con glibc 2.34 o superior. Por esa razón, los scripts del parche acompañan al repositorio, para que cualquier persona pueda reproducir exactamente el mismo entorno. Las alternativas que se evaluaron antes de optar por el parche se discuten en la Sección 2.7.

2.7 Justificación Técnica

La justificación técnica parte de las tecnologías que fija el caso de estudio: el motor de inferencia vLLM, que gestiona el KV cache mediante PagedAttention y planifica la ejecución a nivel de iteración [4]; el modelo Qwen3-8B [2]; y la GPU NVIDIA A100 [3]. Esas tecnologías vienen dadas por la ficha. Esta sección justifica, en cambio, las decisiones que el trabajo sí tomó dentro de ellas: las versiones, el backend de atención, la vía de compatibilidad y la factibilidad de la matriz en memoria.

2.7.1 Versiones y backend de atención

La versión 0.20.0 de vLLM no se eligió por preferencia, sino por una restricción del entorno. Las versiones posteriores distribuyen su stack compilado contra CUDA 13, y el clúster operaba con el driver 535 en el momento de la instalación. La compatibilidad de versiones menores de CUDA permite ejecutar runtimes 12.x sobre drivers 525 o superiores, pero exige un driver 580 o superior para CUDA 13,

en su documentación y su código fuente, disponibles en https://github.com/NixOS/patchelf. Consultado el 24 de octubre de 2026.

de modo que no cruza de la versión mayor 12 a la 13.[5] La versión 0.20.0 con CUDA runtime 12.9 es, por tanto, la más reciente ejecutable con aquel driver sin recurrir a la compatibilidad hacia adelante, el mecanismo de NVIDIA que permite ejecutar CUDA 13 sobre drivers anteriores en GPU de centro de datos mediante el paquete `cuda-compat`.

El driver 575.51.03 observado en `compute-0-2`, recogido en la Tabla 5, no invalida esta decisión. Ese driver sigue por debajo de 580, de modo que la compatibilidad de versiones menores tampoco permitiría ejecutar CUDA 13 con él. Mantener la versión congelada preserva, además, la comparabilidad con las ejecuciones ya realizadas.

vLLM selecciona FlashAttention-2 [24] como backend de atención porque FlashAttention-3 está diseñada para explotar características de la arquitectura Hopper [25], y la A100 pertenece a la arquitectura Ampere. FlashAttention reduce los accesos a la memoria de alto ancho de banda de la GPU durante el cálculo de la atención [9]. Se trata de una limitación de la plataforma y no de una decisión del experimento. Se declara porque afecta al rendimiento absoluto de la fase de prefill, que es la más intensiva en la atención.

Para resolver la incompatibilidad con glibc se evaluaron tres alternativas antes de optar por el parche, recogidas en la Tabla 13. El contenedor oficial queda como plan B porque resuelve el problema sin modificar binarios, pero su tamaño lo hace costoso en el almacenamiento compartido. La compilación desde el código fuente se descartó por su costo en tiempo, y la regresión a una versión anterior, porque no resolvía el problema.

Tabla 13: Alternativas evaluadas para resolver la incompatibilidad con glibc 2.31.

Alternativa	**Motivo del descarte**
Contenedor oficial `vllm/vllm-openai` con enroot/pyxis o Apptainer	Imagen superior a 10 GB sobre NFS compartido; queda como plan B
Compilar vLLM desde el código fuente en el nodo	Varias horas de compilación por cada reinstalación
Fijar una versión anterior de vLLM	No resuelve el problema: la etiqueta incorrecta aparece en varias versiones

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia

[5] NVIDIA, *CUDA Toolkit Release Notes*, tabla de versiones mínimas del driver para la compatibilidad de versiones menores, disponible en `https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/`; y *CUDA Compatibility*, apartado de compatibilidad hacia adelante, disponible en `https://docs.nvidia.com/deploy/cuda-compatibility/forward-compatibility.html`. Consultado el 24 de octubre de 2026.

Fijadas las versiones, queda por justificar que la plataforma puede alojar la matriz experimental y qué rendimiento cabe esperar de ella como máximo. La Sección 2.7.2 lo establece mediante cálculos analíticos.

2.7.2 Techos teóricos y presupuesto de memoria

Los techos teóricos sirven de referencia contra la cual se contrastarán las mediciones. Se calculan con el modelo roofline, que acota el rendimiento de un cálculo por el menor de dos límites: el cómputo máximo del procesador y el ancho de banda de la memoria [17]. Sin esta referencia, una cifra de throughput es un número suelto; con ella, indica qué fracción del límite físico alcanza cada fase.

El punto de partida es el tamaño de los pesos. Qwen3-8B tiene aproximadamente $N = 8{,}2 \cdot 10^9$ parámetros [2], y en BF16 cada parámetro ocupa 2 bytes. La Ecuación 12 obtiene el volumen de los pesos, que equivale a 15,3 GiB.

$$M_{pesos} = N \times 2\ \mathrm{B} = 8{,}2 \cdot 10^9 \times 2\ \mathrm{B} \approx 16{,}4\ \mathrm{GB} \tag{12}$$

En el decode con lote 1, cada token generado exige leer todos los pesos desde la memoria HBM2 de la GPU, y el cómputo por byte leído es bajo. La fase queda entonces limitada por el ancho de banda de memoria, y su techo se obtiene con la Ecuación 13. El cálculo ignora la lectura del KV cache, que con lote 1 es pequeña frente a los pesos, de modo que el techo real es algo inferior.

$$X^{max}_{decode} \approx \frac{BW_{HBM}}{M_{pesos}} = \frac{1555\ \mathrm{GB/s}}{16{,}4\ \mathrm{GB}} \approx 95\ \mathrm{tok/s} \tag{13}$$

En el prefill, en cambio, todos los tokens del prompt se procesan en paralelo y los pesos leídos se reutilizan para cada uno de ellos. La fase queda limitada por el cómputo, a razón de aproximadamente dos operaciones de coma flotante por parámetro y token, una multiplicación y una suma. La Ecuación 14 da su techo con el rendimiento BF16 denso de la A100.

$$X^{max}_{prefill} \approx \frac{P_{BF16}}{2N} = \frac{312 \cdot 10^{12}\ \mathrm{FLOP/s}}{2 \times 8{,}2 \cdot 10^9\ \mathrm{FLOP/tok}} \approx 19\,000\ \mathrm{tok/s} \tag{14}$$

Los dos techos difieren en dos órdenes de magnitud, y esa diferencia es la expresión cuantitativa del argumento central del trabajo: prefill y decode son fases con cuellos de botella distintos. Contrastar las mediciones contra estos techos es lo

que permitirá identificar el cuello de botella dominante de cada fase, como exige el cuarto objetivo específico.

El presupuesto de memoria determina si la configuración más exigente de la matriz cabe en la GPU. Los parámetros de arquitectura se tomaron del archivo `config.json` de la revisión del modelo efectivamente descargada, y no de la documentación, porque es esa revisión la que se ejecuta. El modelo tiene 36 capas y 8 cabezas KV de dimensión 128, gracias a la atención de consultas agrupadas (GQA) [18]. La Ecuación 15 calcula el KV cache por token, donde el factor 2 inicial corresponde a las claves y los valores, y los 2 bytes finales, a la representación BF16.

$$m_{KV} = 2 \times n_{capas} \times n_{KV} \times d_h \times b = 2 \times 36 \times 8 \times 128 \times 2\ \text{B} = 147\,456\ \text{B} = 144{,}0\ \text{KiB} \tag{15}$$

La configuración más exigente de la ficha combina 8192 tokens con 16 secuencias concurrentes. La Ecuación 16 multiplica el KV cache por token por ese volumen, y la Ecuación 17 le suma los pesos. El total, 33,3 GiB, queda dentro de los aproximadamente 36 GiB utilizables de la A100 de 40 GB, pero con un margen estrecho.

$$M_{KV} = 8192 \times 16 \times 147\,456\ \text{B} \approx 18{,}0\ \text{GiB} \tag{16}$$

$$M_{total} = M_{pesos} + M_{KV} \approx 15{,}3\ \text{GiB} + 18{,}0\ \text{GiB} = 33{,}3\ \text{GiB} \tag{17}$$

El cálculo indica que la matriz de la ficha es factible en principio, pero no lo garantiza: el margen restante debe alojar también las activaciones y los grafos CUDA. Por esa razón, el escenario de 8192 tokens con concurrencia 16 en BF16 se identificó en la Sección 2.3 como el hueco prioritario del piloto. Con el método, los instrumentos, el entorno y las cotas analíticas establecidos, el Capítulo III describe la implementación del banco de pruebas y la ejecución de la campaña.

CAPÍTULO III

IMPLEMENTACIÓN, PRUEBAS Y RESULTADOS

3.1 Descripción de la Solución

La solución implementada es un banco de pruebas que mide por separado las fases prefill y decode de Qwen3-8B servido con vLLM en una A100-SXM4-40GB del clúster HPC CEDIA. Se compone del entorno de software congelado y del parche de compatibilidad descritos en la Sección 2.6, del script de medición `bench_pd.sh`, cuyas fases recoge la Tabla 7, y del script de campaña `a100_qwen3_8b.sh`, que la Sección 3.2.2 presenta como respuesta a los problemas observados en la primera ejecución.

Este capítulo informa de la primera ejecución completa sobre la A100 con el modelo objetivo, el trabajo de Slurm 30732. Ese trabajo cubrió las fases 0 a 2 de `bench_pd.sh`: registro del entorno, prueba de humo y barrido sin servidor. Sus resultados son, por tanto, resultados piloto: validan el banco y el método de diferencia de latencias, pero no constituyen todavía la campaña experimental.

3.2 Proceso de Implementación

3.2.1 Ejecución del trabajo 30732

El trabajo 30732 se ejecutó el 24 de octubre de 2026 sobre una A100 completa, sin particiones MIG. La Tabla 14 recoge el entorno registrado por la fase 0 en `entorno.txt`. La carga del nodo al inicio, 37,20 en la ventana de un minuto sobre 256 núcleos, es baja frente a la documentada en la Sección 2.4.2.

Elaborado por: José Andrés Ureta Granda

Fuente: `entorno.txt` y registros del trabajo 30732

Cada configuración del barrido sin servidor se midió con `vllm bench latency`, que crea un motor nuevo en cada invocación, ejecuta tres iteraciones de calentamiento y mide diez. La Tabla 15 desglosa la duración de cada ejecución. La columna de inicialización corresponde al tiempo que vLLM declara para perfilar la memoria, crear el KV cache y calentar el modelo, del cual la mayor parte es compilación con `torch.compile`.

Tabla 14: Entorno registrado en el trabajo 30732.

Elemento	**Valor**
Nodo	`compute-0-1`
Inicio	24 de octubre de 2026, 11:07:24 (UTC−5)
Carga al inicio (1, 5 y 15 min)	37,20; 36,72; 40,91
GPU	NVIDIA A100-SXM4-40GB, 40 960 MiB, límite de 400 W, 1410 MHz máx.
Driver	535.161.08
Software	vLLM 0.20.0, PyTorch 2.11.0+cu129, CUDA 12.9
Modelo	`Qwen/Qwen3-8B`, BF16
Opciones del motor	`max_model_len`=8192, `gpu_memory_utilization`=0,90, caché de prefijos desactivada, chunked prefill activo con 8192 tokens por lote, grafos CUDA y `torch.compile` activos

Tabla 15: Duración de las ejecuciones del trabajo 30732.

Ejecución	**Inicio**	**Inicialización del motor (s)**	**Duración total (min)**	**Medición (s)**
Prueba de humo (L_{in} máx. 2048)	11:32:35	632,8	15,9	—
$L_{in} = 128$, $L_{out} = 1$	12:11:47	1452,4	74,6	0,17
$L_{in} = 128$, $L_{out} = 129$	13:49:08	436,4	28,7	16,95
$L_{in} = 512$, $L_{out} = 1$	14:47:50	248,6	43,1	0,37
$L_{in} = 512$, $L_{out} = 129$	15:55:36	—	Incompleta	—

Elaborado por: José Andrés Ureta Granda

Fuente: Registros `smoke.txt` y `lat_in*_out*_bs1.log` del trabajo 30732

La tabla muestra una desproporción que condiciona el resto de la implementación. Las tres configuraciones completadas sumaron menos de 18 s de medición efectiva en un trabajo de casi cinco horas. La primera compilación de `torch.compile` tardó 1416 s y guardó su resultado en la caché `~/.cache/vllm/torch_compile_cache`, lo que explica que las inicializaciones siguientes bajaran a 436 s y 249 s. Aun así, cada ejecución siguió costando entre 29 y 75 minutos.

La inicialización del motor no explica toda esa duración. En la primera ejecución transcurrieron 38 minutos entre la lectura de la configuración, a las 12:12:25, y la creación del motor, a las 12:50:35, y entre el final de una ejecución y el primer registro de la siguiente pasaron 23 y 30 minutos. La causa de estas esperas no quedó determinada. Los pesos residen en un sistema de archivos NFS4, según el propio registro de vLLM, y su lectura tardó 57 s en la primera ejecución y 10 s en las siguientes, de modo que la lectura de los pesos tampoco basta para explicarlas.

La quinta configuración no llegó a medirse. La telemetría se detiene a las 16:06:11, cuando el motor de esa ejecución aún se estaba inicializando, y ningún registro indica un error.

3.2.2 Rediseño de la ejecución

El trabajo 30732 demostró que, en esta plataforma, el costo dominante no es la medición sino el arranque del motor. Con un arranque de entre 29 y 75 minutos, un barrido que crea un motor por configuración no cabe en un trabajo de Slurm, por largo que sea su límite de tiempo. La decisión de la Sección 2.3 de levantar el servidor una sola vez y recorrer todas las configuraciones contra él pasa así de ser una optimización a ser una condición de viabilidad.

El script `a100_qwen3_8b.sh` materializa esa decisión. Solicita una A100 completa en la partición `gpu-dev`, levanta un único `vllm serve` con las mismas opciones de la Tabla 14, espera hasta 45 minutos a que el endpoint `/health` responda y ejecuta después los experimentos con el cliente `tfgbench` en el mismo nodo, contra `127.0.0.1`. Al ejecutar el cliente y el servidor en el mismo nodo, el TTFT medido no incluye latencia de red. Tras cada experimento, el script guarda las métricas acumuladas de `/metrics`, que sostienen el tercer método de la triangulación de la Sección 2.4.1. Además, reduce el periodo de muestreo de la telemetría de 2 s a 1 s. La Tabla 16 resume los dos experimentos que el script ejecuta.

Elaborado por: José Andrés Ureta Granda

Fuente: `configs/e1_prefill.toml` y `configs/e2_concurrencia.toml`

Tabla 16: Experimentos ejecutados por `a100_qwen3_8b.sh`.

Parámetro	E1: prefill	E2: concurrencia
Longitud de entrada	128, 256, 512, 1024, 2048, 4096 y 7680	1024
Longitud de salida	128	256
Concurrencia	1	1, 2, 4, 8, 16, 32 y 64
Solicitudes medidas por punto	30 (más 3 de calentamiento)	256 (más 4 de calentamiento)
Métricas principales	TTFT	TPOT, ITL y throughput

En ambos experimentos los prompts son secuencias aleatorias de identificadores de token de longitud exacta, generadas con semilla 0 y sin prefijos compartidos, y las solicitudes se envían con temperatura 0 e `ignore_eos`. La longitud máxima de E1 es 7680 y no 8192, porque la suma de entrada y salida no puede superar el `max_model_len` de 8192. En E2, con 64 secuencias de 1280 tokens, el KV cache ocupa unos 11,3 GiB y cabe en la memoria disponible sin expulsiones, según el presupuesto de la Sección 3.3.3.

El primer lanzamiento de este script, el trabajo 31157 del 27 de octubre, no llegó a medir. El servidor no respondió en los 45 minutos de espera, su registro quedó vacío y la GPU no llegó a ocupar memoria.

3.3 Pruebas y Validación

3.3.1 Prueba de humo y verificación del dispositivo

La prueba de humo cargó Qwen3-8B y generó 64 tokens de texto coherente en español a partir de un prompt sobre la inferencia de modelos de lenguaje. Su tiempo total, 13,77 s, incluye la primera ejecución del modelo y no se usa como medida de rendimiento. La prueba cumple su función: confirma que el entorno parcheado de la Sección 2.6.2 carga el modelo y genera texto antes de comprometer el resto del trabajo.

El dispositivo se verificó con `nvidia-smi -L`, que identifica una A100-SXM4-40GB completa y no una instancia MIG, conforme a la precaución de la Sección 2.1. El backend de atención seleccionado fue FlashAttention-2, como anticipa la Sección 2.7.1.

3.3.2 Estabilidad de las mediciones

La Tabla 17 resume la dispersión de las diez iteraciones medidas en cada configuración. El coeficiente de variación no supera el 0,6 %, y el rango completo de cada configuración es inferior a 1 ms. Con lote 1 y sin servidor, la medición es, por tanto, muy repetible dentro de un mismo trabajo.

Tabla 17: Dispersión de la latencia en las diez iteraciones medidas.

Configuración	Media (ms)	Desv. típica (ms)	CV (%)	Mín. (ms)	Máx. (ms)
L_{in} = 128, L_{out} = 1	17,087	0,099	0,58	16,943	17,205
L_{in} = 512, L_{out} = 1	37,207	0,138	0,37	36,984	37,396
L_{in} = 128, L_{out} = 129	1695,221	0,306	0,02	1694,844	1695,809

Elaborado por: José Andrés Ureta Granda
Fuente: Archivos `lat_in*_out*_bs1.json` del trabajo 30732

Las iteraciones de calentamiento confirman la necesidad de descartarlas. La primera de cada configuración tardó 25,2 s, 14,9 s y 33,5 s, respectivamente, entre 9 y 1500 veces más que la media de las iteraciones medidas, porque incluye la captura de grafos y la carga de kernels pendientes. Incluirla en la media habría invalidado la medición, como prevé el requerimiento RNF3.

La baja dispersión no garantiza, sin embargo, la reproducibilidad entre trabajos. Las diez iteraciones de cada configuración se ejecutaron en menos de 20 s, dentro de un mismo motor y con una carga del nodo estable. La variación entre trabajos, nodos y versiones del driver, que la Sección 2.4.2 identifica como la principal amenaza, solo podrá estimarse con repeticiones en trabajos distintos.

3.3.3 Contraste del presupuesto de memoria

Los registros de vLLM permiten contrastar el presupuesto analítico de la Sección 2.7.2. Los pesos ocuparon 15,27 GiB, frente a los 15,3 GiB de la Ecuación 12. Con `gpu_memory_utilization` de 0,90, vLLM reservó 16,32 GiB para el KV cache, equivalentes a 118 832 tokens, y 2,44 GiB para los grafos CUDA. El cociente entre ambas cifras del KV cache es de 147 456 bytes por token, que coincide exactamente con la Ecuación 15.

El contraste revela que la configuración más exigente de la ficha no cabe en la GPU con estas opciones. Con 8192 tokens por solicitud, el KV cache disponible admite 14,51 secuencias simultáneas, según el propio registro de vLLM, y no las

16 que exige la ficha. En las ejecuciones posteriores, la reserva subió a 16,85 GiB y 122 672 tokens, que admiten 14,97 secuencias, cifra que sigue por debajo de 16. El presupuesto de la Ecuación 17 era correcto en sus términos, pero omitía los 2,44 GiB de los grafos CUDA y el 10 % de la memoria que vLLM deja sin reservar.

Este resultado cierra el primero de los huecos señalados en la Sección 2.3. La combinación de 8192 tokens y concurrencia 16 solo sería ejecutable aumentando `gpu_memory_utilization`, desactivando los grafos CUDA con `--enforce-eager` o aceptando que el planificador ponga solicitudes en espera. Cualquiera de las tres opciones altera lo que se mide, y por esa razón E1 limita la entrada a 7680 tokens con una sola solicitud, y E2 limita la entrada a 1024 tokens.

3.4 Resultados

Los resultados de esta sección proceden exclusivamente del barrido sin servidor del trabajo 30732, con lote $B = 1$. Se calculan con el método de diferencia de latencias de la Sección 2.4.1 y se contrastan con los techos teóricos de la Sección 2.7.2.

3.4.1 Fase de prefill

La Tabla 18 aplica las Ecuaciones 7 y 8 a las dos longitudes de entrada medidas. La columna de MFU (*model FLOPs utilization*) expresa el throughput de prefill como fracción del techo de cómputo [20], con la Ecuación 18, donde P_{BF16} es el rendimiento BF16 denso de la A100 y N el número de parámetros.

$$\text{MFU} = \frac{2N \times X_{prefill}}{P_{BF16}} \tag{18}$$

Tabla 18: Latencia y throughput de prefill con lote 1.

L_{in} **(tokens)**	$T_{prefill}$ **(ms)**	$X_{prefill}$ **(tok/s)**	**MFU (%)**
128	17,09	7 491	39,4
512	37,21	13 761	72,3

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia a partir del trabajo 30732

El throughput de prefill casi se duplica al cuadruplicar la entrada, mientras que la latencia solo se multiplica por 2,2. Ese comportamiento indica que una parte de $T_{prefill}$ no depende de la longitud de entrada. Si se supone una relación lineal, $T_{prefill} = a + b\,L_{in}$, los dos puntos medidos determinan la pendiente de la Ecuación 19 y una ordenada en el origen $a \approx 10{,}4$ ms.

$$b = \frac{T_{prefill}(512) - T_{prefill}(128)}{512 - 128} = \frac{37{,}21 - 17{,}09}{384}\ \text{ms} \approx 0{,}0524\ \text{ms/token} \quad (19)$$

El inverso de esa pendiente equivale a 19 085 tok/s, prácticamente igual al techo de 19 000 tok/s de la Ecuación 14. Sin embargo, esta coincidencia no permite afirmar que el prefill alcance el 100 % del techo de cómputo en el margen. Lo más probable es que el modelo lineal sea demasiado simple. Con 128 tokens, las multiplicaciones de matrices son pequeñas y aprovechan peor los Tensor Cores que con 512, de modo que la eficiencia crece con L_{in} y la pendiente entre dos puntos queda subestimada. Además, la Ecuación 14 ignora las operaciones de la atención, que crecen con el cuadrado de la longitud.

Con dos puntos no es posible separar el costo fijo de la variación de la eficiencia. Por esa razón, el experimento E1 mide siete longitudes entre 128 y 7680 tokens. El dato robusto de este piloto es la cota inferior: con 512 tokens, el prefill ya alcanza el 72 % del techo de cómputo, lo que confirma que es una fase limitada por el cómputo.

3.4.2 Fase de decode

La Ecuación 9 aplicada a la entrada de 128 tokens da un TPOT de 13,11 ms, equivalente a 76,3 tok/s por secuencia. Ese valor representa el 80 % del techo de 95 tok/s de la Ecuación 13. La Ecuación 20 expresa la misma relación como MBU (*model bandwidth utilization*), la fracción del ancho de banda de la memoria que se aprovecha al leer los pesos en cada paso [21].

$$\text{MBU} = \frac{M_{pesos}/\text{TPOT}}{BW_{HBM}} = \frac{16{,}4\ \text{GB}/13{,}11\ \text{ms}}{1555\ \text{GB/s}} = \frac{1251\ \text{GB/s}}{1555\ \text{GB/s}} \approx 80\,\% \quad (20)$$

Un MBU del 80 % indica que el decode con lote 1 está limitado por el ancho de banda de la memoria y opera cerca de su techo. El 20 % restante corresponde a la lectura del KV cache, que la Ecuación 13 ignora, y al sobrecosto por paso del motor, que no se puede separar con estos datos. La desviación típica de 0,31 ms sobre 1695 ms de la Tabla 17 implica que el TPOT se conoce con una incertidumbre inferior a 0,01 ms.

El TPOT de 512 tokens de entrada no se obtuvo porque la ejecución con $L_{out} = 129$ no se completó. Por tanto, el piloto no permite observar cómo crece el TPOT con el contexto, que es el efecto esperado de la lectura del KV cache.

3.4.3 Contraste entre fases

Los dos resultados anteriores cuantifican la asimetría que motiva el trabajo. Procesar los 128 tokens del prompt costó 17,09 ms, mientras que generar un único token costó 13,11 ms. En la práctica, el prefill procesó 128 tokens en un tiempo apenas un 30 % mayor que el que el decode necesita para generar uno solo. La Tabla 19 resume ambas fases frente a sus techos.

Tabla 19: Comparación de las fases frente a sus techos teóricos ($L_{in} = 128$, $B = 1$).

Fase	**Recurso limitante**	**Medido (tok/s)**	**Techo (tok/s)**	**Utilización**
Prefill	Cómputo	7 491 ($L_{in} = 128$) 13 761 ($L_{in} = 512$)	19 000	MFU 39 % MFU 72 %
Decode	Ancho de banda	76,3	95	MBU 80 %

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia a partir del trabajo 30732 y de la Sección 2.7.2

Las dos fases difieren en dos órdenes de magnitud en throughput por secuencia, como anticipaba el análisis de la Sección 2.7.2. Con lote 1, el decode aprovecha una fracción mayor de su recurso limitante que el prefill de entradas cortas. Sin embargo, su throughput absoluto es 98 veces menor con 128 tokens y 180 veces menor con 512.

3.4.4 Telemetría de la GPU

La GPU ocupó como máximo 36 799 MiB, el 90 % de su memoria, conforme al valor de `gpu_memory_utilization`. En reposo, con la memoria vacía, la potencia media fue de 55,2 W. La potencia máxima, 268 W, se registró durante el calentamiento y la medición de la configuración con 129 tokens de salida, que es la única que mantuvo la GPU ocupada durante más de unos segundos.

La telemetría del trabajo 30732 no permite estimar la energía por token con la Ecuación 11. Con un muestreo cada 2 s, las mediciones de prefill, que duraron menos de medio segundo, no contienen ninguna muestra propia, y la de decode contiene unas ocho. Esta limitación motivó la reducción del periodo a 1 s en `a100_qwen3_8b.sh`. En E2, cuyos puntos duran varios minutos, ese periodo bastará para integrar la energía, pero en E1 seguirá siendo insuficiente.

3.4.5 Alcance de los resultados piloto

Los resultados de esta sección validan el método de diferencia de latencias y los techos teóricos, pero tienen un alcance limitado. Proceden de un único trabajo en un único nodo, con lote 1, dos longitudes de entrada y una de salida. Además, solo aplican el primero de los tres métodos de triangulación, porque la fase en línea no llegó a ejecutarse. Las métricas de cliente y las de `/metrics`, que permitirían contrastarlos, corresponden a los experimentos E1 y E2.

3.5 Cronograma y Presupuesto

[Presentar la planificación de las fases de ejecución del proyecto y el detalle de los costos asociados a hardware, licencias, servicios u otros recursos.]

CAPÍTULO IV

CONCLUSIONES Y RECOMENDACIONES

4.1 Conclusiones

[Redactar conclusiones basadas en los resultados obtenidos y en el cumplimiento de los objetivos planteados.]

4.2 Recomendaciones

[Proponer mejoras, optimizaciones o ampliaciones del proyecto, fundamentadas en el análisis realizado.]

4.3 Trabajos Futuros

[Plantear líneas de desarrollo o investigación que puedan derivarse del proyecto implementado.]

[Incluya todas las referencias bibliográficas utilizadas en el documento. Deben estar citadas según normas IEEE y gestionadas mediante un gestor bibliográfico como Mendeley o Zotero.]

BIBLIOGRAFÍA

[1] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, 2024. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[2] Qwen Team, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[3] NVIDIA Corporation, "NVIDIA A100 tensor core GPU datasheet," https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf, 2021, consultado el 16 de octubre de 2026.

[4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Koblenz, Germany: ACM, 2023, arXiv:2309.06180.

[5] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou *et al.*, "A survey of large language models," 2023. [Online]. Available: https://arxiv.org/abs/2303.18223

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Curran Associates, Inc., 2017, arXiv:1706.03762.

[7] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*.

Santa Clara, CA: USENIX Association, 2024. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/agrawal

[8] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-Based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, 2022, pp. 521–538. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/yu

[9] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-Awareness," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022, arXiv:2205.14135.

[10] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proceedings of Machine Learning and Systems 6 (MLSys 2024)*, Santa Clara, CA, 2024, best Paper Award. arXiv:2306.00978.

[11] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022.

[12] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, and I. Stoica, "AlpaServe: Statistical multiplexing with model parallelism for deep learning serving," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 663–679.

[13] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proceedings of Machine Learning and Systems*, vol. 5, 2023.

[14] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, "Llumnix: Dynamic scheduling for large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 173–191.

[15] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution

of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[16] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024.

[17] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[18] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023, pp. 4895–4901. [Online]. Available: https://aclanthology.org/2023.emnlp-main.298/

[19] D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha *et al.*, "A study of BFLOAT16 for deep learning training," 2019. [Online]. Available: https://arxiv.org/abs/1905.12322

[20] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts *et al.*, "PaLM: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023. [Online]. Available: https://www.jmlr.org/papers/v24/22-1144.html

[21] M. Agarwal, A. Qureshi, N. Sardana, L. Li, J. Quevedo, and D. Khudia, "LLM inference performance engineering: Best practices," Databricks Blog. [En línea]. Disponible en: https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices, Oct. 2023, consultado el 24 de octubre de 2026.

[22] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: Simple Linux utility for resource management," in *Job Scheduling Strategies for Parallel Processing (JSSPP 2003)*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2003, pp. 44–60.

[23] N. J. Smith and T. Kluyver, "PEP 600 – future 'manylinux' platform tags for portable Linux built distributions," Python Enhancement Proposals. [En línea].

Disponible en: https://peps.python.org/pep-0600/, 2019, consultado el 24 de octubre de 2026.

[24] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024, arXiv:2307.08691.

[25] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024, pp. 68 658–68 685.

GLOSARIO

[Defina los términos técnicos relevantes utilizados en el documento.]

Término o sigla 1 *[Definición técnica breve y clara.]*

Término o sigla 2 *[Definición técnica breve y clara.]*

ANEXO A

TÍTULO DEL ANEXO A

[Incluya una descripción breve del contenido del anexo. Ejemplo: documentación técnica, diagramas, configuraciones, evidencias o instrumentos aplicados.]

- *[Documentación técnica.]*
- *[Manuales de usuario y administración.]*
- *[Evidencias de pruebas.]*
- *[Autorizaciones y validaciones de la entidad receptora.]*
- *[Repositorio de versionamiento (enlace).]*
- *[Formatos de entrevistas y encuestas aplicadas.]*